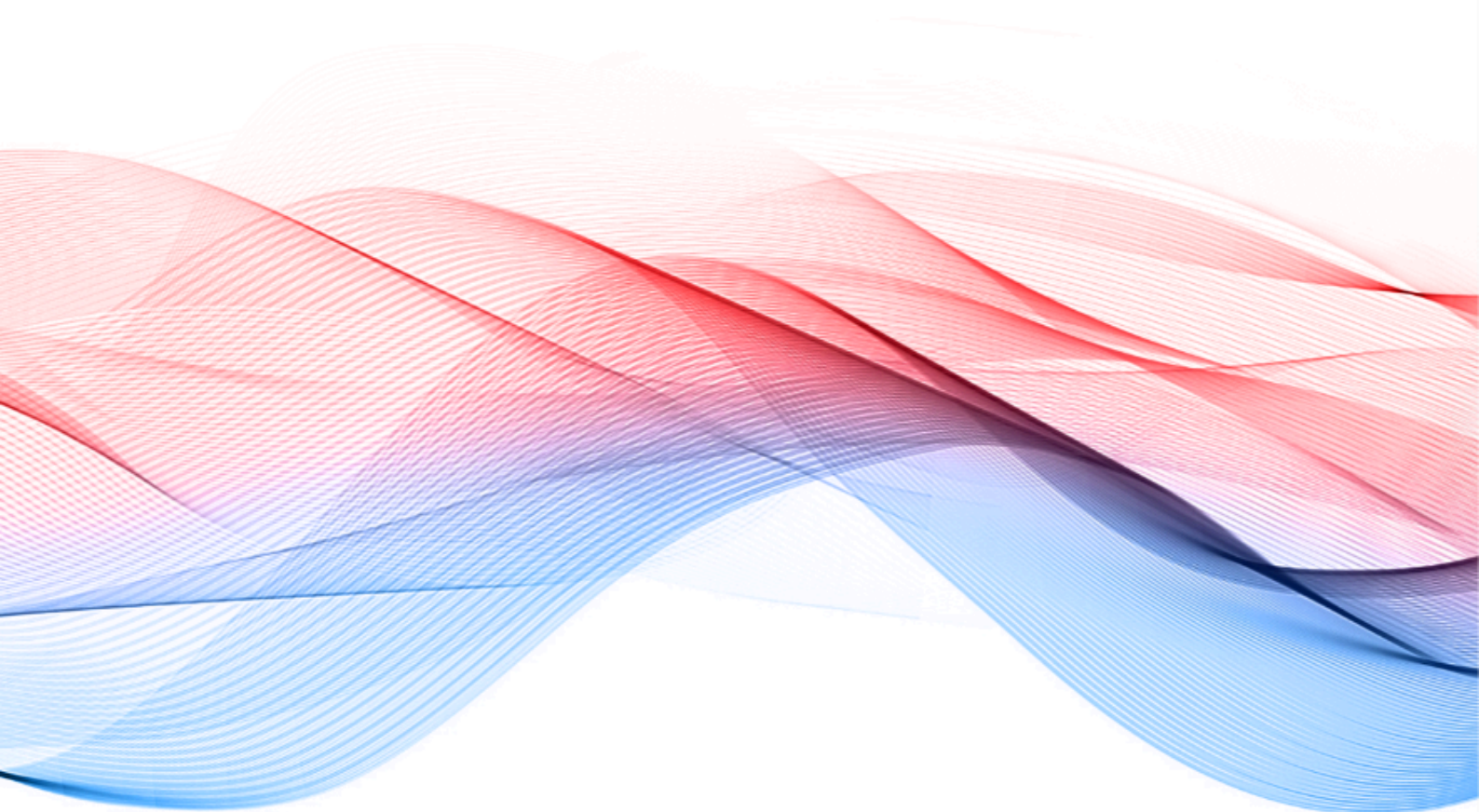


拓斯达科技

X5-Python 库 API 说明书

v3.1 | 2025-10



文档说明

本文档的所有内容（包括但不限于文字、图片、图表、音频、视频等）均受到中华人民共和国著作权法和其他相关法律法规的保护。未经广东拓斯达科技股份有限公司（以下简称“本公司”）的书面许可，任何个人或组织均不得以任何形式对本文档中的内容进行复制、传播、展示、发布、修改、翻译、销售、许可或以其他方式侵犯本公司的知识产权。

本公司保留随时更新和修改本文档和手册的权利，恕不另行通知。请注意，本文档和手册中的产品名称、标识和服务标志均属于本公司的商标或注册商标。未经本公司的书面许可，任何个人或组织均不得使用这些商标或注册商标。

如果您发现本文档中存在任何侵权行为，请立即与本公司联系，我们将依法予以处理。

广东拓斯达科技股份有限公司

修订记录

修订日期	手册版本	修订内容
2025/10	v3.1	添加警告内容，说明部分接口的使用事项 新增获取所有系统状态 get_all_system_state 接口 新增获取当前运动完成状态 get_cur_move_state 接口 与 xapi_py_v3.2.5 版本配套升级
2025/07	v3.0	首次发布，与 xapi_py_v3.1.x 版本配套

目录

文档说明	2
修订记录	3
前序	10
使用对象	10
注意事项	10
安全	11
快速开始	12
库介绍与安装	12
库介绍	12
库安装条件	12
使用 pip 安装库	12
查看库版本	12
使用	13
代码	13
运行结果	13
自定义数据类型说明(结构体)	14
点位数据类型	14
Pose	14
Point	14
Joint	20
MovPointAdd	20
系统状态数据类型	23
SystemState	23
AllSystemState	24
程序信息结构体	26
ProgramInfo	26
实时运动模式状态	27
RGMRobState	27
通信连接	28

连接管理	28
get_ip_list 扫描 IP 地址	28
connect 创建与机器人的连接	29
disconnect 断开与机器人的连接	30
get_version 获取版本信息	31
状态管理	32
系统状态	32
set_remote 设置远程状态	32
set_system_mode 设置运行模式	33
enable_servo 设置使能状态	34
reset 复位机器人报警	35
abort 停止机器人所有动作	36
get_system_state 获取机器人系统状态	37
get_system_alarm_info 获取机器人报警信息	38
get_all_system_state 获取机器人系统状态	39
位置状态	40
get_cpoint 获取当前笛卡尔点位（用户坐标系）	40
get_wpoint 获取当前笛卡尔点位（世界坐标系）	41
get_cjoint 获取当前关节点位	42
运动控制	43
运动驱动	43
movl 直线轨迹运动	43
movj 快速轨迹运动	44
movc 圆弧轨迹运动	45
jumpl 门型直线轨迹运动	46
jump 门型快速轨迹运动	47
jogl_rel 沿笛卡尔坐标系方向增量运动	48
jogl 沿笛卡尔坐标系方向持续运动	49
jogj_rel 沿关节坐标系方向增量运动	50
jogj 沿关节坐标系方向持续运动	51
movsj 关节坐标系下单轴运动	52
servoj 在线实时控制关节位置	53

servol 在线实时控制 TCP 位置	54
实时引导运动 RGM	55
rgm_init RGM 初始化	56
rgm_destroy RGM 销毁	56
rgm_start RGM 启动	56
rgm_stop RGM 停止	57
rgm_servoj RGM 实时关节位置运动	57
rgm_servol RGM 实时 TCP 位置运动	58
RGM 示例	59
运动配置	61
set_speed 设置机器人全局速度	61
get_speed 获取机器人全局速度	62
get_cur_move_state 获取当前运动指令完成状态	63
wait_move_done 等待机器人运动完成	65
程序管理	66
前台程序	66
run 运行程序	66
pause 暂停程序或运动	67
resume 恢复程序或运动	67
stop 停止程序或运动	68
set_path_back_type 设置停止再启动返回方式	68
start 开始运行程序	69
前台程序示例	69
后台程序	72
daemon_run 运行或恢复后台程序	72
daemon_pause 暂停后台程序	72
daemon_stop 停止后台程序	72
后台程序示例	73
程序模块	74
get_module_name_list 获取模块名列表	74
load_module 加载模块	74
get_compile_state 获取系统程序编译状态	74

get_program_info 获取当前模块的程序信息	75
程序模块示例	75
IO 管理	77
IO 设定	77
set_do 设置数字量输出状态	77
get_do 获取数字量输出状态	78
set_ao 设置模拟量输出电压值	79
get_ao 获取模拟量输出电压值	80
get_di 获取数字量输入状态	81
get_ai 获取模拟量输入电压值	82
get_ei 获取编码器输入值	83
IO 强制	84
set_force_di 强制设置数字量输入状态	84
get_force_di 获取数字量输入强制状态	85
reset_force_di 复位数字量输入强制状态	86
set_force_ai 强制设置模拟量输入电压值	87
get_force_ai 获取模拟量输入强制状态	88
reset_force_ai 复位模拟量输入强制状态	89
reset_all_force_io 复位所有端口的强制状态	90
IO 命令	91
wait_set_do_done 等待数字量输出状态执行完成	91
变量管理	92
数值变量	92
set_hr 设置 HR 变量	92
get_hr 获取 HR 变量	93
set_mbr 设置 MBR 变量	94
get_mbr 获取 MBR 变量	95
位置变量	96
set_pr 设置 PR 变量	96
get_pr 获取 PR 变量	97
set_jpr 设置 JPR 变量	98
get_jpr 获取 JPR 变量	99

参数管理	100
坐标系	100
set_ufno 设置当前用户坐标系序号	100
get_ufno 获取当前用户坐标系序号	101
set_tfno 设置当前工具坐标系序号	102
get_tfno 获取当前工具坐标系序号	103
set_uf 设置用户坐标系数据	104
get_uf 获取用户坐标系数据	105
set_tf 设置工具坐标系数据	106
get_tf 获取工具坐标系数据	107
set_bf 设置基坐标系数据	108
get_bf 获取基坐标系数据	109
calc_uf 标定用户坐标系	110
calc_tf 标定工具坐标系	114
calc_2p_tf 目标点标定工具坐标系	117
cnvrt_j 笛卡尔点位逆解求关节点位	119
cnvrt_p 关节点位正解求笛卡尔点位	120
cnvrt_p_to_p 笛卡尔点位坐标系转换	121
工艺应用	122
视觉工艺	122
vision_static_cnvrt 计算静态视觉转换后的点位数据	122
vision_dynamic_cnvrt 计算动态视觉转换后的点位数据	123
高级接口	124
CMD 命令	124
execute_lua 执行 lua 字符串命令(TODO)	124
wait_cmd_send_done 等待命令下发完成	126
高级调试	127
enable_debug_output 是否输出调试信息	127
enable_read_message_thread 是否开启字符串读取线程	128
get_lua_message 获取 Lua 反馈字符串	129
get_last_error 获取最后一次 API 错误	130
logger_get_recent_errors 获取最近多条错误日志	131

logger_shutdown 释放日志缓存资源	132
export_api_log_file 导出 API 日志文件	133

附录	134
-----------	------------

Ubuntu-Linux 实时环境配置	134
下载实时内核	134
下载依赖	134
编译安装内核	134
显卡在实时内核下安装驱动解决方法：	135
日志功能	139
运行日志	139
例行程序	140
上位机控制机器人自动运行指定程序	140
上位机控制机器人运行点位	147
报错详情	153

前序

使用对象

本手册适用于具有 Python 语言编程基础的工程开发人员。

注意事项

- 兼容性：本说明书适用于 Python 库 xapi_py_v3.2.5 及以后版本使用。
- 版本对应：xapi_py_v3.2.5 版本仅完全支持 X5 控制器 v5.5.1 版本并兼容 v5.4.x 版本。
- 链接跳转：文档中的倾斜蓝色字体均支持点击跳转到对应位置：如[快速开始](#)。
- 接口说明：所有接口的说明以本文档为准，其他地方的描述可能存在更新延迟的情况。
- 参数说明：部分接口或结构体包含众多控制器相关参数，请参考本文档的相关说明或结合其他说明书（如《机器人控制系统——使用说明书》）以获取详细的使用指导。
- 接口更新：对于新接口的拓展，我们通常会优先保留原有接口(若有)，以支持旧接口的用户。然而，随着版本的迭代，我们可能会逐步剔除旧接口，尤其在发生重大不兼容版本迭代时。因此，我们建议用户逐步适配新接口，以确保在未来版本中能够充分利用新功能。在接口更新时，请查阅版本发布说明，以获取详细信息和适配建议。
- 安全注意：请确保在与控制器进行通信时采取安全措施，防止潜在的风险和安全漏洞。
- 命令队列状态：建议在调用接口前检查命令队列的状态，并采取相应措施以避免在队列满时出现命令下发或执行失败的情况。
- 工作线程：GUI 主线程一般用于数据展示和人机交互，推荐使用工作线程完成接口调用和逻辑操作。
- 最快刷新时间：接口内部需一定执行时间，建议最快刷新时间不低于 20ms。
- 线程优先级：对不同线程设置不同线程刷新时间。如对于高优先级线程(如点位刷新)，可考虑设置 20ms；对于低优先级线程(如 IO 获取)，可考虑设置 150ms 或 200ms 左右。
- 线程数量：合理规划线程数量，避免线程滥用。在队列满时，频繁调用接口可能导致命令排队下发的等待问题。
- 资源释放：在使用完 SDK 资源后，请及时释放相应资源，以避免脏数据、内存泄漏。
- 多版本管理：如需同时使用不同版本的 SDK，建议采用适当的版本管理策略，避免不同版本的 SDK 之间的不兼容性。

安全

本书中所示的注意事项为本产品相关的事项。客户使用本产品自行编入机器人作为最终产品时，所产生的相关安全问题，请客户自行考虑。

快速开始

库介绍与安装

库介绍

xapi 是 Python 的一个与机器人相关的库，可对机器人控制器进行编程控制，可进行操作包括与机器人通讯、IO 交互、机器人运动、机器人程序运行等。

xapi 是第三方库，不是 Python 安装程序自带的库，需要额外安装。

库安装条件

假设你已经安装了 Python 3.10 版本（建议使用 3.10 版本），且可以正常使用 pip 工具。

假设你已经从厂商获得库文件(如 xapi-3.2.5-cp310-cp310-win_amd64.whl)。

使用 pip 安装库

以管理员身份运行 cmd 命令行窗口，可以使用 cd 命令来切换到库文件所在的目录：

```
cd C:\Users\Administrator\Downloads
```

或是进入安装包的存放目录下，执行终端命令

再使用 pip install 来安装本地库，即命令行窗口中输入如下命令：

```
pip install xapi-3.2.5-cp310-cp310-win_amd64.whl
```

等待 pip 完成安装过程，直到命令行窗口出现如下信息：

```
Successfully installed xapi
```

查看库版本

在命令行窗口中输入如下命令：

```
pip list
```

可查看结果如下：

```
xapi 3.2.5
```

使用

下面我们使用 xapi 进行简单的机器人数据交互。

代码

```
import xapi.api as x5
# 关闭高级调试信息输出
x5.enable_debug_output(0)
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}")
# 获取并打印版本信息
version = x5.get_version(handle)
print(f"版本信息: {version}")
# 获取指定数字量输出端口状态
data = x5.get_do(handle, index=0)
print(f"DO0 端口状态: {data}\n")
# 删除连接
x5.disconnect(handle)
```

运行结果

XEnableDebugOutput:<Disable>

控制器句柄号: 0

版本信息: xapipy:3.2.5(x5.5), API:5.3.3[3], T:3.1.8.0, M:2.2.4.0, P:3.1.8.0, E:3.205.0.6,
K:xos_mc36_l_v1.4.0, PLC:4.1.6.0, Upgrade:xmotion_v5.4.8.gamma, Config:2.1.0

DO0 端口状态: 1

自定义数据类型说明(结构体)

点位数据类型

Pose

位姿数据类型使用：

```
import xapi.api as x5
# 创建 Pose 数据类型变量
pos = x5.Pose(x=0, y=0, z=0, a=0, b=0, c=0, e1=0, e2=0, e3=0)
# 如果是七轴协作机器人，R1 目标值为 30，则定义如下
pos = x5.Pose(x=0, y=0, z=0, a=0, b=0, c=0, e1=30, e2=0, e3=0)
```

数据类型说明：Pose 位姿数据类型，包含位姿数据，描述的是刀具中心点(TCP)在用户坐标系下的位置信息，其中包含刀具中心点的位置(x, y, z)、刀具中心点的姿态(a, b, c)、附加轴位置(e1、e2、e3)。当机型为七轴协作机器人时，E1 轴表示机器人 R1 数据，不做附加轴使用。

Point

笛卡尔点位数据类型使用：

```
import xapi.api as x5
# 创建 Point 数据类型变量
pos = x5.Pose(x=0, y=0, z=0, a=0, b=0, c=0, e1=0, e2=0, e3=0)
point = x5.Point(pose=pos, uf=0, tf=0, cfg=(0,0,0,1))
# 如果是七轴协作机器人，R1 目标值为 30，则定义如下
pos = x5.Pose(x=0, y=0, z=0, a=0, b=0, c=0, e1=30, e2=0, e3=0)
point = x5.Point(pose=pos, uf=0, tf=0, cfg=(0,0,0,1))
```

警告

若需设置机器人运动时的用户和工具坐标系序号，用户应直接修改目标点位的 uf 和 tf 参数。在机器人执行运动时，将基于目标点位的用户和工具坐标系序号移动到目标点位，并自动切换至相应的用户和工具序号。

数据类型说明：Point 笛卡尔点位数据类型，包含笛卡尔位置数据，描述的是 Pose 位姿数据类型的位姿数据(pose)、所使用的用户坐标系编号(uf)、工具坐标系编号(tf)、以及机器人本体的形态值(cfg)。

$\{ \underline{\text{pose}}, \underline{\text{uf}}, \underline{\text{tf}}, \underline{\text{cfg}} \}$ 位姿 用户 工具 形态
--

位姿数据(pose): Pose 型, 包含 x、y、z、a、b、c、e1、e2、e3 的值。当机型为七轴协作机器人时, E1 轴表示机器人 R1 数据, 不做附加轴使用。

用户编号(uf): int 型, 由世界坐标系或用户坐标系的编号指定, 范围 UF0~UF16。

工具编号(tf): int 型, 由法兰盘坐标系或工具坐标系的编号指定, 范围 TF0~TF16。

形态数据(cfg): int 型, 指机器人主体机构的位姿, 即{CF1, CF2, CF3, CFX}。其中需要指定各个轴的旋转数(CF1、CF2、CF3)和关节配置(CFX)。

1. 旋转数(CF1、CF2、CF3): 表示机器人关节轴顺时针或者逆时针旋转了几周到达同一位置。(-1: -539°~-180°; 0: -179°~179; 1: 180°~539°)。

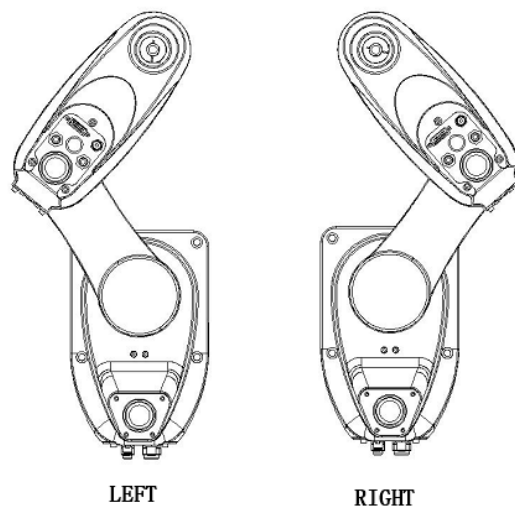
- **SCARA 机器人旋转数:** CF1 表示: J1 轴的旋转数; CF2 表示: J2 轴的旋转数; CF3 表示: J4 轴的旋转数。
- **Delta 机器人旋转数:** CF1 表示: J1 轴的旋转数; CF2 表示: J2 轴的旋转数; CF3 表示: J4 轴的旋转数。
- **六轴机器人旋转数:** CF1 表示: J1 轴的旋转数; CF2 表示: J4 轴的旋转数; CF3 表示: J6 轴的旋转数。
- **六轴协作机器人旋转数:** CF1 表示: J1 轴的旋转数; CF2 表示: J4 轴的旋转数; CF3 表示: J6 轴的旋转数。
- **七轴协作机器人旋转数:** CF1 表示: J1 轴的旋转数; CF2 表示: J5 轴的旋转数; CF3 表示: J7 轴的旋转数。

一般来说, Scara 机器人只有 J4 轴可以设置旋转数, 而 6 轴机器人的 J1、J4、J6 轴都可以设置旋转数。而六轴协作机器人、七轴协作机器人各自对应的轴也都可以设置旋转数。

2. 关节配置(CFX)

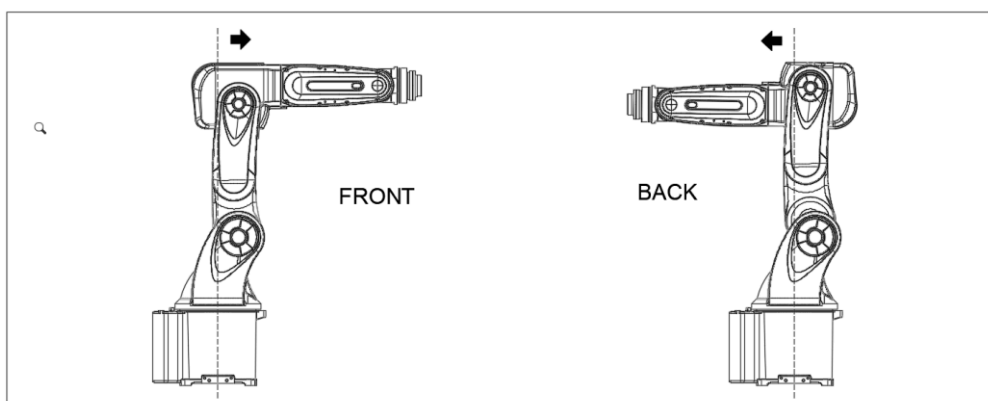
表示机械手腕和机臂的配置, 指定机械手腕和机臂的控制点相对于控制面位于那一侧。控制面上控制点相互重叠时, 机器人位于特殊点(奇异点)。特殊点上由于存在着无限种基于笛卡尔坐标值的形态, 机器人不能操作。

SCARA 机器人关节配置如下:

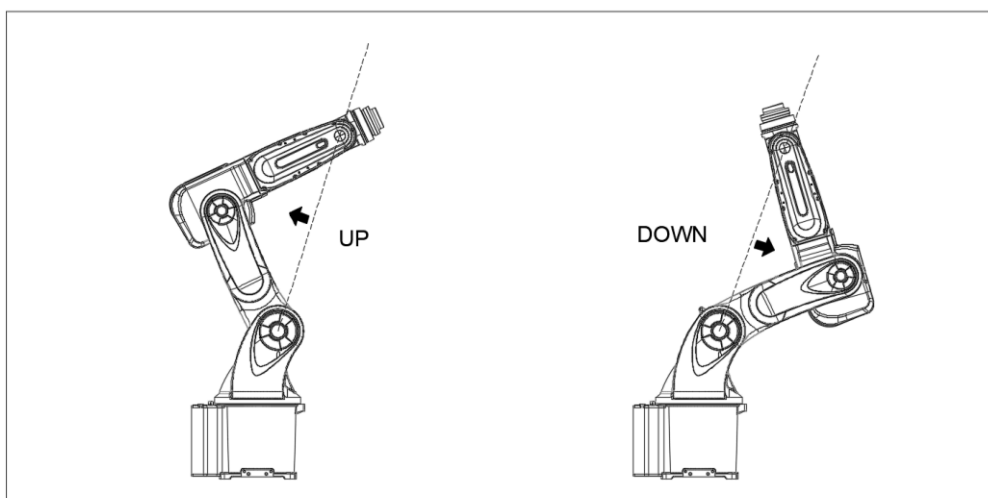


CFX	J2 轴关节
0	LEFT
1	RIGHT

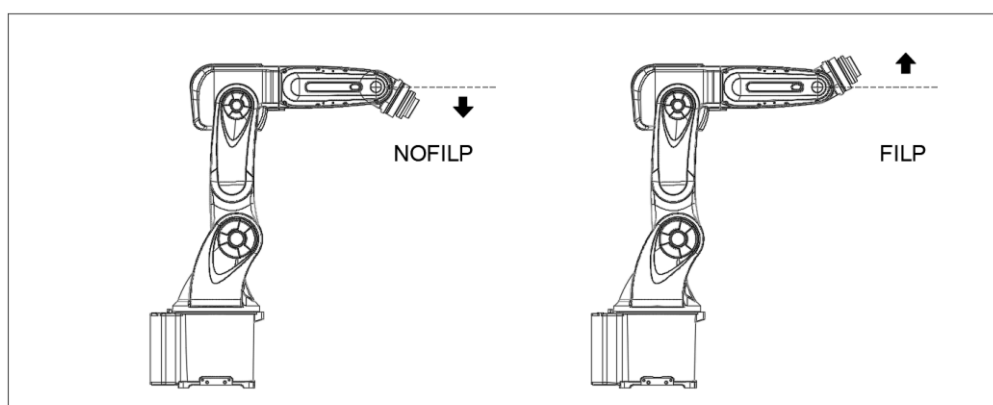
六轴机器人关节配置如下： J1 轴关节配置(FRONT/BACK)



J3 轴关节配置(UP/DOWN)



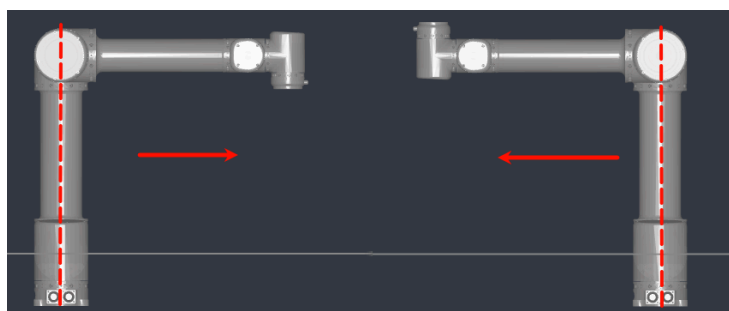
J5 轴关节配置(NOFILP/FILP)



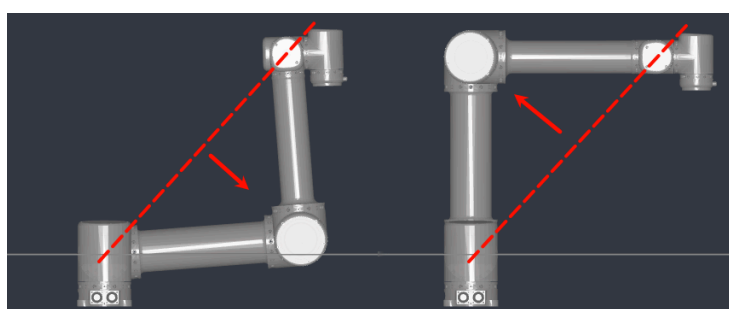
CFX	J1 轴关节	J3 轴关节	J5 轴关节
0	FRONT	UP	NOFLIP
1	BACK	UP	NOFLIP
2	FRONT	DOWN	NOFLIP
3	BACK	DOWN	NOFLIP
4	FRONT	UP	FLIP
5	BACK	UP	FLIP
6	FRONT	DOWN	FLIP
7	BACK	DOWN	FLIP

- 六轴协作机器人关节配置如下：

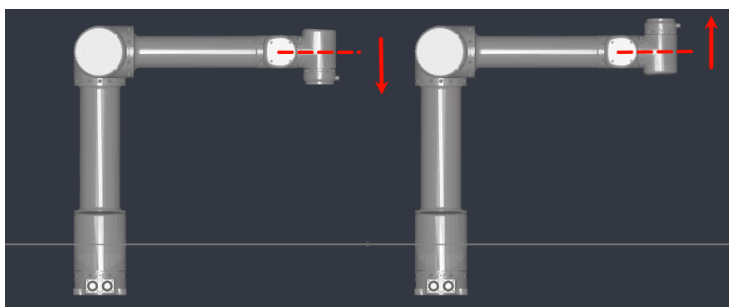
J1 轴关节配置(FRONT/BACK)



J3 轴关节配置(UP/DOWN)



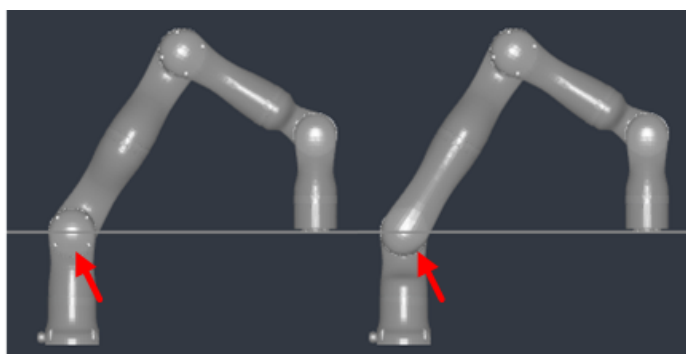
J5 轴关节配置(FILP/NOFILP)



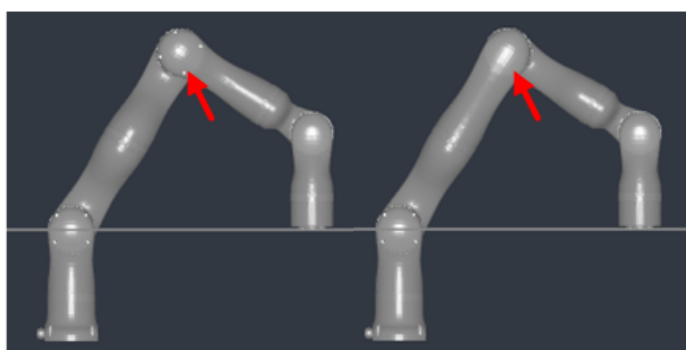
CFX	J1 轴关节	J3 轴关节	J5 轴关节
0	FRONT	UP	NOFLIP
1	BACK	UP	NOFLIP
2	FRONT	DOWN	NOFLIP
3	BACK	DOWN	NOFLIP
4	FRONT	UP	FLIP
5	BACK	UP	FLIP
6	FRONT	DOWN	FLIP
7	BACK	DOWN	FLIP

- 七轴协作机器人关节配置如下：

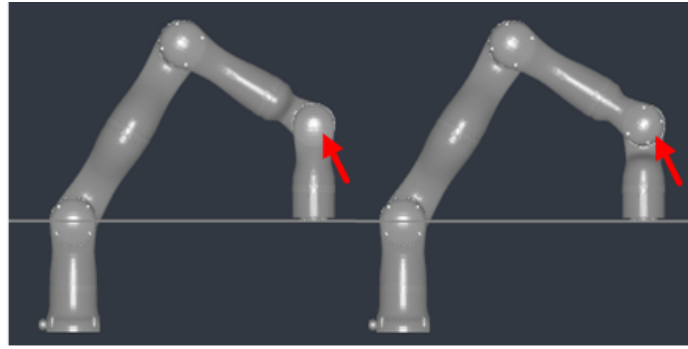
J1 轴关节配置(FRONT/BACK)



J4 轴关节配置(UP/DOWN)



J6 轴关节配置(NOFILP/FILP)



CFX	J1 轴关节	J4 轴关节	J6 轴关节
0	FRONT	UP	NOFLIP
1	BACK	UP	NOFLIP
2	FRONT	DOWN	NOFLIP
3	BACK	DOWN	NOFLIP
4	FRONT	UP	FLIP
5	BACK	UP	FLIP
6	FRONT	DOWN	FLIP
7	BACK	DOWN	FLIP

Joint

关节位数据类型使用：

```
import xapi.api as x5
# 创建 Joint 数据类型变量
joint = x5.Joint(j1=0, j2=0, j3=0, j4=0, j5=0, j6=0, e1=0, e2=0, e3=0)
# 如果是七轴协作机器人，J7 轴目标值为 30，则定义如下
joint = x5.Joint(j1=0, j2=0, j3=0, j4=0, j5=0, j6=0, e1=30, e2=0, e3=0)
```

数据类型说明：关节位数据类型，包含关节位置数据，描述的是机器人当前位姿下各关节的坐标位置值，其中包含机器人 j1~j6 各关节的位置坐标值(j1、j2、j3、j4、j5、j6)，3 个附加轴位置坐标值(e1、e2、e3)。当机型为七轴协作机器人时，E1 轴表示机器人 J7 轴位置数据，不做附加轴使用。

MovPointAdd

附加动作数据类型使用：

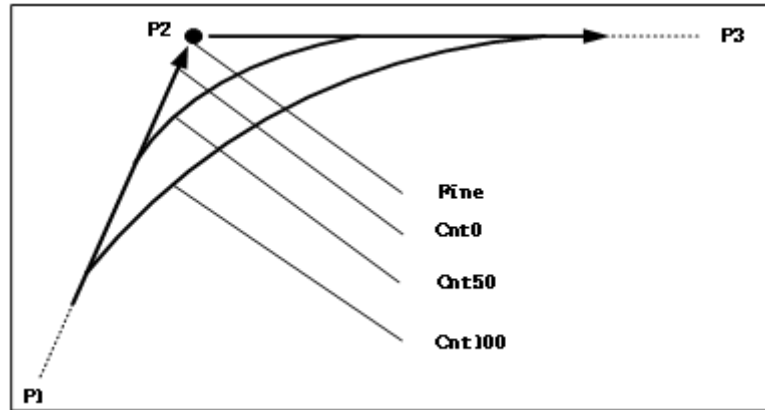
```
import xapi.api as x5
# 创建 MovPointAdd 数据类型变量
add_data = x5.MovPointAdd()
add_data.vel = 100 # 速度百分比 (1~100)
add_data.cnt = 0 # 定位类型(-1:Fine; 0~100:cnt)
add_data.acc = 100 # 加速度百分比 (1~200)
add_data.dec = 100 # 减速度百分比 (1~200)
add_data.offset = -2 # 位置补偿(-2:不使用; -1:offset 常量)
add_data.toffset = -2 # 刀具补偿(-2:不使用; -1:toffset 常量)
add_data.vision_num = -1 # 视觉工艺(-1:不使用; 0~7:视觉工艺补偿编号)
add_data.track_num = -1 # 跟踪工艺(-1:不使用; 0~7:跟踪工艺编号)
add_data.wrist = 0 # 腕部动作(0:不使用; 1~:腕部动作类型)
add_data.inc = 0 # 增量动作(0:不使用; 1~:使用)
add_data.search = 0 # 搜索动作(0:不使用; 1~:使用)
add_data.rtcp = 0 # rtcp 设置(0:不使用; 1~:使用)
#offset 位姿数据(仅 offset 常量时有效)
add_data.offset_data = x5.Pose(0, 0, 0, 0, 0, 0)
#toffset 位姿数据(仅 toffset 常量时有效)
add_data.toffset_data = x5.Pose(0, 0, 0, 0, 0, 0)
```

数据类型说明：

MovPointAdd 附加动作数据类型，包含运动指令所有的附加动作设置数据。

速度百分比(vel)：int 型，设置机器人运行当前运动指令的速度百分比，值域范围为 1~100。

定位类型(cnt): int 型, 设置当前运动指令的定位类型为-1, 则按 Fine 类型定位; 当 cnt=0~100 时, 按 cnt 类型定位。定位类型是指运动指令执行过程中的动作结束方式, 分为 Fine 和 cnt 两种。定位类型 Fine, 是指机器人在到达当前目标位置停止后, 再向着下一个目标位置运动。定位类型 cnt, 是指机器人靠近或者到达当前目标位置后, 直接向着下一个目标位置运动。



cnt=0 表示机器人最靠近目标位置但不停止, 开始向着下一个目标位置运动;

cnt=100 表示机器人靠近目标位置不减速开始向着下一个目标位置运动, 即实际不到达目标点 P2, 而走一段平滑的圆弧到达下一个运动指令的目标点 P3。

加速度百分比(acc): int 型, 设置当前运动指令的加速度百分比, 值域范围为 1~200。

减速度百分比(dec): int 型, 设置当前运动指令的减速度百分比, 值域范围为 1~200。

位置补偿(offset): int 型, 设置当前运动指令的位置补偿模式,

当 offset=-2, 为不使用位置补偿; 当 offset=-1, 为 offset 常量位置补偿, 其中位置补偿在 offset_data 数据类型变量中设置;

工具补偿(toffset): int 型, 设置当前运动指令的工具补偿模式,

当 toffset=-2, 不使用位置补偿; 当 toffset=-1, 为 toffset 常量工具补偿, 其中工具补偿在 toffset_data 结构体变量中设置;

视觉工艺编号(vision_num): int 型,

当 vision_num=-1 时, 不使用视觉补偿;

当 vision_num=0~7 时, 选择视觉工艺编号 0~7 进行视觉补偿。

跟踪工艺编号(track_num): int 型,

当 track_num=-1 时, 不使用跟踪补偿;

当 track_num=0~7 时, 选择跟踪工艺编号 0~7 进行跟踪补偿。

腕部动作(wrist): int 型,

当 wrist=0 时, 不使用腕部动作;

当 wrist>=1 时, 使用腕部动作。一般为六轴机器人在直线运动或圆弧运动过腕部奇异点时使用该指令。

增量动作(inc): int 型,

当 inc=0 时，不使用增量动作；

当 inc = 1 时，使用增量动作。增量动作指令，是指将该指令中的位置数据所记录的值作为当前位置的增量偏移值而使机器人移动。即相对运动指令，相对移动量为位置数据所记录的值。

搜索动作(search)：int 型，

当 search=0 时，不使用搜索动作；

当 search=1 时，使用搜索动作。搜索动作指令，是指机器人向置数据中所记录的目标位置移动过程中，当搜索条件满足时，机器人放弃运动到目标点，立即停止并执行下一行程序指令；

若条件未满足，机器人将运动到目标点再并执行下一行程序指令。**外部刀具(rtcp)：**int 型，

当 rtcp=0 时，不使用外部刀具；

当 rtcp= 1 时，使用外部刀具。详见《X5 机器人控制系统——使用说明书》。

位置补偿偏移点位参数(offset_data)：Pose 型，包含 x、y、z、a、b、c、e1、e2、e3。如需设置此参数，需设置 offset 为-1 常量。

工具补偿偏移点位参数(toffset_data)：Pose 型，包含 x、y、z、a、b、c、e1、e2、e3。如需设置此参数，需设置 toffset 为-1 常量。

系统状态数据类型

SystemState

系统状态数据类型说明：

参数名	说明
run	运行状态(1: 运行, 2: 暂停, 3: 停止)
mode	系统当前模式(2: 自动模式, 3: 手动模式, 5: 调试模式, 100: 自动命令模式)
in_pos	到位信号(0: 运动中, 1: 轴到位)
alarm	错误状态(0: 正常)
remote	远程状态(0: 本地, 1: 远程)
link	通信链接状态(0: 链接失败, 1: 链接成功)
enable	系统使能状态(0: 下使能, 1: 上使能)
cmd_que	运动队列满标识位(0: 队列空, 1: 队列有余)
rgm_state	系统 RGM 模式标识位: (0: 非 RGM 模式, 1: RGM 模式)

数据类型使用：

```
import xapi.api as x5
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 获取系统状态
state = x5.get_system_state(handle)
print(f"系统运行状态: {state.run}")
print(f"系统模式: {state.mode}")
print(f"系统到位信号: {state.in_pos}")
print(f"系统错误状态: {state.alarm}")
print(f"系统远程状态: {state.remote}")
print(f"系统通信连接状态: {state.link}")
print(f"系统使能状态: {state.enable}")
print(f"系统运动队列满标识位: {state.cmd_que}")
print(f"系统 RGM 模式标识位: {state.rgm_state}\n")
```

AllSystemState

系统状态数据类型说明：

参数名	说明
link	(int):通信状态 0:通信异常/断开, 1:通信正常
enable	(int): 使能状态 0:未上电, 1:已上电
alarm	(int): 报警状态 0:无报警, >0:发生报警个数
mode	(int): 运行模式 2:自动模式, 3:手动模式, 5:调试模式, 100:自动 CMD 模式
state	(int): 运行状态 1:运行, 2:暂停, 3:停止
in_pos	(int): 轴到位信号 0:轴运动 1:轴停止
remote	(int): 远程状态 0:本地 1:远程
cmd_queue	(int): 命令队列标识位 0: 队列空, 1: 队列有余, 2: 队列满
rgm_state	(int): 系统 RGM 模式标识位 0:非 RGM 模式, 1:RGM 模式
speed	(int): 全局速度 1~100%
uf_no	(int): 用户坐标系序号 0~16
tf_no	(int): 工具坐标系序号 0~16
user_point	(Point): 当前用户坐标系下的笛卡尔点位
world_point	(Point): 当前世界坐标系下的笛卡尔点位
joint	(Joint): 当前关节点位
di_list	(unsigned char * 32): 数字输入状态(按 bit 位顺序存放, bit0~bit247; bit0 表示 DI0, 以此类推) (0:低电平, 1:高电平)
do_list	(unsigned char * 32): 数字输出状态(按 bit 位顺序存放, bit0~bit247; bit0 表示 DO0, 以此类推) (0:低电平, 1:高电平)
ei_list	(int * 8): 编码器输入

数据类型使用：

```
import xapi.api as x5
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 获取所有系统状态
all_system_state = x5.get_all_system_state(handle)
print(f"系统通信连接状态: {all_system_state.link}")
print(f"系统使能状态: {all_system_state.enable}")
print(f"系统错误状态: {all_system_state.alarm}")
print(f"系统模式: {all_system_state.mode}")
```

```

print(f"系统运行状态: {all_system_state.state}")
print(f"系统到位信号: {all_system_state.in_pos}")
print(f"系统远程状态: {all_system_state.remote}")
print(f"系统运动队列满标识位: {all_system_state.cmd_queue}")
print(f"系统 RGM 模式标识位: {all_system_state.rgm_state}")
print(f"系统全局速度: {all_system_state.speed}")
print(f"系统当前用户坐标系序号: {all_system_state.uf_no}")
print(f"系统当前工具坐标系序号: {all_system_state.tf_no}")
print(f"系统当前用户坐标系下点位: {all_system_state.user_point}")
print(f"系统当前世界坐标系下点位: {all_system_state.world_point}")
print(f"系统当前关节点位: {all_system_state.joint}")
print(f"系统所有 DI 数据")
for i in range(0, 31): # 例如 DI0~DI7 值为 3, 即二进制值为 00000011, 即 DI0 与 DI1 值为 1
    print(f"DI{i*8}~DI{i*8+7}: {all_system_state.di_list[i]}")
print(f"系统所有 DO 数据")
for i in range(0, 31): # 例如 DO0~DO7 值为 3, 即二进制值为 00000011, 即 DO0 与 DO1 值为 1
    print(f"DO{i*8}~DO{i*8+7}: {all_system_state.do_list[i]}")
print(f"系统所有 EI 数据")
for i in range(0, 8): # 编码器输入 int 值
    print(f"EI{i}: {all_system_state.ei_list[i]}")

```

程序信息结构体

ProgramInfo

程序信息数据类型说明：

参数名	说明
name	程序名称(char * 64)
priority	优先级 (0:前台程序, 1~5:后台程序)
type	程序类型 (0:前台程序, 1:静态任务[后台自动运行], 2:动态任务[随主程序启动后运行], 3:指令任务[指令调用])
state	运行状态 (1:运行, 2:暂停, 3:停止)

数据类型使用：

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
try:
    # 创建连接
    handle = x5.connect(ip)
    print(f"\n 控制器句柄号: {handle}\n")
    # 获取当前模块的程序信息
    result = x5.get_program_info(handle)
    print(f"\n 当前模块的程序信息: {result[0]}\n")
    t.sleep(1)
except x5.RobException as rex:
    print(f"报错接口: {rex.api_name}")
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
```

实时运动模式状态

RGMRobState

实时运动模式状态数据类型说明：

参数名	说明
state	机器人实时引导运动状态 (0:停止 1:准备 2:运行中 3:暂停)
point	机器人当前笛卡尔位置 (Point 型)
joint	机器人当前关节位置 (Joint 型)

数据类型使用：

- 详见 [RGM 示例](#)

通信连接

连接管理

get_ip_list | 扫描 IP 地址

扫描网段内所有允许连接的目标机器人，返回其网口号和 IP 地址

原型

```
def get_ip_list() -> List[Dict[str, int | str]]:
```

返回值

- `ip_list (List[Dict[str, Union[int, str]]])`: 所有 IP 地址字符串的列表
 - 其中元素格式: { “net_no” :网口号, “ip” :IP 地址}

执行条件

- 无

示例

```
import xapi.api as x5
# 获取 IP 地址
ip_list = x5.get_ip_list()
print("IP 数量:", len(ip_list))
for i in range(0, len(ip_list)):
    print(f"目标[{i}] 网口: {ip_list[i]['net_no']}, IP: {ip_list[i]['ip']}")
# 输出:
# 目标[0] 网口号: 1, IP 地址: 168.168.80.30
# 目标[1] 网口号: 2, IP 地址: 168.168.100.85
# 目标[2] 网口号: 2, IP 地址: 168.168.100.40
# 目标[3] 网口号: 2, IP 地址: 168.168.100.30
```

connect | 创建与机器人的连接

创建与机器人的 TCP 连接，并返回句柄号。句柄号是调用其它接口时的验证标识，需妥善保存。单个应用程序最多创建 64 个连接。

原型

```
def connect(ip, link_timeout, retry_num=2, link_timeout=5) -> int:
```

参数

- `ip (str)`: 机器人 IP 地址。
- `link_timeout (int)`: 连接超时时间。默认值 2 秒
- `retry_num (int)`: 超时重连次数。默认值 5 次

返回值

- `handle (int)`: 机器人句柄号

执行条件

- 无

示例

```
import xapi.api as x5

# 机器人 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"机器人句柄号: {handle}")

# 输出:
# 机器人句柄号: 0
```

💡 备注

对于 X5 控制器，NET0 网口默认 IP 地址为 192.168.1.6，NET1 网口默认 IP 地址为 172.18.11.5

disconnect | 断开与机器人的连接

断开与机器人的 TCP 连接。请手动断开不再使用的连接，避免连接数量超限

原型

```
def disconnect(handle) -> None:
```

参数

- `handle (int)`: 机器人句柄号

执行条件

- 无

示例

```
import xapi.api as x5

# 机器人 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
# ...中间代码
# 断开连接
x5.disconnect(handle)
```

get_version | 获取版本信息

获取机器人的各项版本信息

⚠ 警告

用户界面程序通过 API 库连接机器人时，每次获取版本信息可避免引用库版本错误问题。

原型

```
def get_version(handle) -> str:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `version (str)`: 版本信息字符串

执行条件

- 无

示例

```
import xapi.api as x5

# 机器人 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
# 获取并打印版本信息
version = x5.get_version(handle)
print(f"版本信息: {version}")
# 输出:
# 版本信息: xapipy:3.1.11(x5.4), API:5.1.18[3], T:3.1.8.0, M:2.2.4.0, P:3.1.8.0,
E:3.205.0.6, K:xos_mc36_l_v1.4.0, PLC:4.1.6.0, Upgrade:xmotion_v5.4.6.gamma,
Config:2.1.0
```

状态管理

系统状态

set_remote | 设置远程状态

设置机器人的远程状态。

原型

```
def set_remote(handle, is_remote) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `is_remote (bool)`: 远程状态(True: 远程模式,False: 本地模式)

执行条件

- 机器人处于非调试模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# set_remote
x5.set_remote(handle, True)
t.sleep(1)
x5.set_remote(handle, False)
t.sleep(1)
```

set_system_mode | 设置运行模式

设置机器人运行模式。可选模式包括自动、手动、调试、自动命令模式。

原型

```
def set_system_mode(handle, mode) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `mode (int)`: 机器人运行模式
 - `SYSTEM_MODE_AUTO/2`: 自动模式
 - `SYSTEM_MODE_MANUAL/3`: 手动模式
 - `SYSTEM_MODE_DEBUG/5`: 调试模式
 - `SYSTEM_MODE_AUTOCMD/100`: 自动命令模式

执行条件

- 机器人处于停止状态

示例

```
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 切换为自动模式
x5.set_system_mode(handle, x5.SYSTEM_MODE_AUTO)
t.sleep(1)
# 切换为自动命令模式
x5.set_system_mode(handle, x5.SYSTEM_MODE_AUTOCMD)
t.sleep(1)
# 切换为手动模式
x5.set_system_mode(handle, x5.SYSTEM_MODE_MANUAL)
t.sleep(1)
```

💡 备注

1. 模式切换时序详见设置使能状态接口时序图
2. 机器人手动模式下仅支持单个运动接口执行，运动期间不允许发送运动指令。
3. 机器人自动模式下仅支持执行程序接口，不允许执行运动接口。
4. 机器人手动或自动命令模式下仅支持执行运动接口，不允许执行程序接口。

enable_servo | 设置使能状态

设置机器人使能状态

原型

```
def enable_servo(handle, is_servo_on) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `is_servo_on (bool)`: (True:上使能,False:下使能)

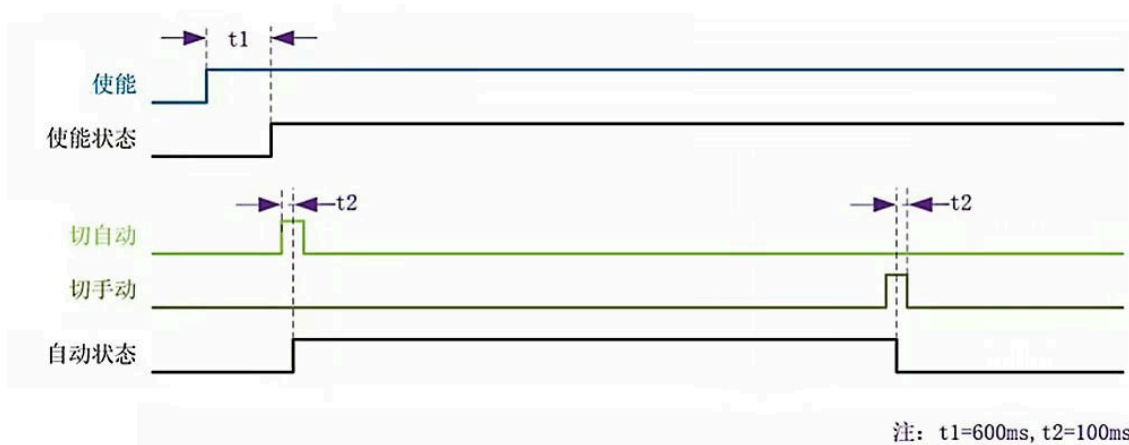
执行条件

- 机器人处于本地模式(非远程模式)

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 系统上使能
x5.enable_servo(handle, True)
t.sleep(1)
# 系统下使能
x5.enable_servo(handle, False)
t.sleep(1)
```

时序图:



reset | 复位机器人报警

尝试机器人当前的报警状态复位。

⚠ 警告

机器人报警时，用户应立即查看报警信息，并决定是否复位或返回起点重新执行运动。

原型

```
def reset(handle) -> None:
```

参数

- `handle (int)`: 机器人句柄号

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 系统报警复位
x5.reset(handle)
t.sleep(1)
```

abort | 停止机器人所有动作

停止机器人当前所有动作（包括程序或运动）

原型

```
def abort(handle) -> None:
```

参数

- `handle (int)`: 机器人句柄号

执行条件

- 任意模式

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 停止系统当前所有动作
x5.abort(handle)
# 等待机器人减速停止运动完成
x5.wait_move_done(handle)
t.sleep(1)
```

💡 备注

1. 调用 stop 和 abort 接口停止机器人运动时，机器人会需要时间来进行减速停止。
2. 如有单独线程在持续下发运动，调用 abort 接口后，请自行处理停止线程下发。
3. 接口中止程序后，将无法停止再启动程序！

get_system_state | 获取机器人系统状态

获取机器人的系统状态信息

原型

```
def get_system_state(handle) -> SystemState:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `state (SystemState)`: 系统状态。详情查看数据结构 [SystemState](#) 结构体

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 获取系统状态
state = x5.get_system_state(handle)
print(f"系统状态: \n{state}\n")
t.sleep(1)
```

get_system_alarm_info | 获取机器人报警信息

获取机器人的报警信息

⚠ 警告

本接口涉及数据库访问，不建议频繁调用。

原型

```
def get_system_alarm_info(handle) -> List[Dict[str, int | str]]:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `error_index (List[Dict[str, Union[int, str]]])`: 报警信息队列

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 获取系统状态
state = x5.get_system_state(handle)
if(state.alarm == 1):
    # 获取系统报警码
    error = x5.get_system_alarm_info(handle)
    print(f"报警数量: {len(error)}")
    print(f"报警码队列: {error}\n")
t.sleep(1)
# 打印输出
# 控制器句柄号: 0

# 报警数量: 1
# 报警码队列: [{'level': 0, 'type': '2', 'code': '201', 'content': '系统急停',
'reason': 'NULL', 'solution': '检查电气线路解除急停。'}]
```

get_all_system_state | 获取机器人系统状态

获取机器人的系统状态信息

原型

```
def get_all_system_state(handle) -> AllSystemState:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `system_state (AllSystemState)`: 所有系统状态数据。详见 [AllSystemState](#) 结构体

执行条件

- 任意模式

示例

```
import xapi.api as x5
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 获取所有系统状态
all_system_state = x5.get_all_system_state(handle)
print(f"系统通信连接状态: {all_system_state.link}")
# ...
print(f"系统模式: {all_system_state.mode}")
print(f"系统运行状态: {all_system_state.state}")
print(f"系统全局速度: {all_system_state.speed}")
print(f"系统当前用户坐标系序号: {all_system_state.uf_no}")
print(f"系统当前用户坐标系下点位: {all_system_state.user_point}")
print(f"系统所有 DI 数据")
for i in range(0, 31): # 例如 DI0~DI7 值为 3, 即二进制值为 00000011, 即 DI0 与 DI1 值为 1
    print(f"DI{i*8+0}: {all_system_state.di_list[i] % 2}")
    print(f"DI{i*8+1}: {all_system_state.di_list[i] // 2}")
    print(f"DI{i*8+2}: {all_system_state.di_list[i] // 4}")
    print(f"DI{i*8+3}: {all_system_state.di_list[i] // 8}")
    print(f"DI{i*8+4}: {all_system_state.di_list[i] // 16}")
    print(f"DI{i*8+5}: {all_system_state.di_list[i] // 32}")
    print(f"DI{i*8+6}: {all_system_state.di_list[i] // 64}")
    print(f"DI{i*8+7}: {all_system_state.di_list[i] // 128}")
```

位置状态

get_cpoint | 获取当前笛卡尔点位（用户坐标系）

获取当前用户与工具坐标系下的笛卡尔点位

原型

```
def get_cpoint(handle) -> Point:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `point (Point)`: 笛卡尔点位数据

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# get_cpoint
point = x5.get_cpoint(handle)
print(f"当前用户下的笛卡尔点位数据: {point}\n")
t.sleep(1)
```

get_wpoint | 获取当前笛卡尔点位（世界坐标系）

获取当前世界坐标下的笛卡尔点位

原型

```
def get_wpoint(handle) -> Point:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `point (Point)`: 笛卡尔点位数据

执行条件

- 任意模式

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# get_wpoint
point = x5.get_wpoint(handle)
print(f"当前世界下的笛卡尔点位数据: {point}\n")
t.sleep(1)
```

get_cjoint | 获取当前关节点位

获取当前机器人的关节点位

原型

```
def get_cjoint(handle) -> Joint:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `joint (Joint)`: 关节点位数据

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# get_cjoint
joint = x5.get_cjoint(handle)
print(f"当前关节点位数据: \n{joint}\n")
t.sleep(1)
```

运动控制

运动驱动

⚠ 警告

1. 当机器人执行运动时，会基于目标点位的**用户坐标系**和**工具坐标系序号**移动到目标点位，并自动切换至相应的**用户和工具序号**。
2. 当机器人进行连续运动时，需使用附加动作中的 `cnt` 参数进行速度过渡，否则可能出现运动卡顿。
3. 当机器人执行运动时，若调用 `stop`、`abort` 接口，或出现机器人报警，机器人将停止并清除已下发的运动。需注意，若调用 `stop` 或 `abort` 接口后，另有线程继续下发运动，机器人将执行后续下发的运动。

movl | 直线轨迹运动

在笛卡尔坐标系下，以直线轨迹运动到目标点

原型

```
def movl(handle:int, point:Point, additional_data:Optional[MovPointAdd] = None) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `point (Point)`: 目标笛卡尔点位
- `additional_data (MovPointAdd,optional)`: 附加动作参数

执行条件

- 机器人上使能，手动或自动命令模式

示例

```
import xapi.api as x5
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 沿笛卡尔空间运动到目标点
```

```

x5.movel(handle,point=x5.Point((200,200,0,0,0,0,0,0,0),0,0,(0,0,0,1)))
x5.wait_move_done(handle)
# 沿笛卡尔空间运动到目标点
pos = x5.Point((250,200,0,0,0,0,0,0,0),0,0,(0,0,0,1))
# 如果是七轴协作机器人，R1 目标值为 30，则定义如下
# pos = x5.Point((250,200,0,0,0,0,30,0,0),0,0,(0,0,0,1))
add_data = x5.MovPointAdd(vel=50, cnt=0, acc=50, dec=50, offset =-1,
offset_data=(10,0,0,0,0,0,0,0,0))
x5.movel(handle, pos, add_data)
x5.wait_move_done(handle)

```

movj | 快速轨迹运动

在空间坐标系下，以快速轨迹运动到目标点位

原型

```

def movj(handle:int, point:Point | Joint, additional_data:Optional[MovPointAdd] =
None) -> None:

```

参数

- `handle (int)`: 机器人句柄号
- `point (Point|Joint)`: 目标笛卡尔或关节点位数据
- `additional_data (MovPointAdd, optional)`: 附加动作参数

执行条件

- 机器人上使能，手动或自动命令模式

示例

```

import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 沿关节运动到笛卡尔目标点
x5.movj(handle,point=x5.Point((200,200,0,0,0,0,0,0,0),0,0,(0,0,0,1)))
x5.wait_move_done(handle)
# 沿关节运动到笛卡尔目标点
pos = x5.Point((250,200,0,0,0,0,0,0,0),0,0,(0,0,0,1))

```

```
# 如果是七轴协作机器人，R1 目标值为 30，则定义如下
# pos = x5.Point((250,200,0,0,0,0,30,0,0),0,0,(0,0,0,1))
Data = (10,0,0,0,0,0,0,0)
add_data = x5.MovPointAdd(vel=50, acc=50, offset=-1, offset_data = Data)
x5.movj(handle, pos, add_data)
x5.wait_move_done(handle)
# 沿关节运动到关节目标点
pos = x5.Joint(10,0,0,0,0,0,0,0)
add_data = x5.MovPointAdd(vel=50, acc=50)
x5.movj(handle, pos, add_data)
x5.wait_move_done(handle)
```

movc | 圆弧轨迹运动

在笛卡尔坐标系下，经中间点以圆弧轨迹运动到目标点位。轨迹是以起始点、中间点和目标点确定的圆弧。如果设置圆心角，运动将在圆心角对应的目标点位处停止

原型

```
def movc(handle: int, middle_point: Point, target_point: Point, central_angle:
Optional[float] = None, additional_data: Optional[MovPointAdd] = None) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `middle_point (Point)`: 中间点
- `target_point (Point)`: 目标点
- `central_angle (float, optional)`: 圆心角，单位为角度。默认为 0，此时执行标准 MovC 运动。
- `add_data (MovPointAdd, optional)`: 附加动作(可选)

执行条件

- 机器人上使能，手动或自动命令模式

示例

```
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 沿关节运动到笛卡尔目标点
x5.movj(handle, point=x5.Point((200,200,0,0,0,0,0,0),0,0,(0,0,0,1)))
```

```

x5.wait_move_done(handle)
# 进行 movc 运动
middle_point=x5.Point((220,220,0,0,0,0,0,0),0,0,(0,0,0,1))
target_point=x5.Point((240,200,0,0,0,0,0,0),0,0,(0,0,0,1))
central_angle = 180
add_data = x5.MovPointAdd(vel=50, cnt=0, acc=50, dec=50, offset =-1,
offset_data=(10,0,0,0,0,0,0,0))
x5.movc(handle,middle_point,target_point,central_angle,add_data)
x5.wait_move_done(handle)

```

jump | 门型直线轨迹运动

在笛卡尔坐标系下，以门型直线轨迹运动到点位

原型

```

def jump(handle: int, point: Point, jump_data: List[float], additional_data:
Optional[MovPointAdd] = None) -> None:

```

参数

- `handle (int)`: 机器人句柄号
- `point (Point)`: 笛卡尔点位
- `jump_data (List[float])`: 门型运动参数，长度为 3 的浮点数列表
- `additional_data (MovPointAdd, optional)`: 附加动作参数

执行条件

- 机器人上使能，手动或自动命令模式

示例

```

import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 沿关节运动到笛卡尔目标点
x5.movj(handle, point=x5.Point((200,200,0,0,0,0,0,0),0,0,(0,0,0,1)))
x5.wait_move_done(handle)
# 沿笛卡尔空间 jump 门型运动到笛卡尔目标点
pos = x5.Point((220,220,0,0,0,0,0,0),0,0,(0,0,0,1))

```

```

jump_data=[10,10,400]
x5.jump1(handle, pos, jump_data)
x5.wait_move_done(handle)
# 沿笛卡尔空间 jump 门型运动到笛卡尔目标点(带附加动作)
pos = x5.Point((240,200,0,0,0,0,0,0,0),0,0,(0,0,0,1))
Data = (10,0,0,0,0,0,0,0,0)
add_data = x5.MovPointAdd(vel=50, acc=50, offset=-1, offset_data = Data)
jump_data=[10,10,-10]
x5.jump1(handle, pos, jump_data, add_data)
x5.wait_move_done(handle)

```

jump | 门型快速轨迹运动

在笛卡尔坐标系下，以门型快速轨迹运动到目标点位

原型

```

def jump(handle: int, point: Point, jump_data: List[float], additional_data:
Optional[MovPointAdd] = None) -> None:

```

参数

- `handle (int)`: 机器人句柄号
- `point (Point)`: 笛卡尔点位
- `jump_data (List[float])`: 门型运动参数，长度为 3 的浮点数列表
- `additional_data (MovPointAdd, optional)`: 附加动作参数

执行条件

- 机器人上使能，手动或自动命令模式

示例

```

import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 沿关节运动到笛卡尔目标点
x5.movj(handle, point=x5.Point((200,200,0,0,0,0,0,0,0),0,0,(0,0,0,1)))
x5.wait_move_done(handle)
# 沿笛卡尔空间 jump 门型运动到笛卡尔目标点

```

```

pos = x5.Point((220,220,0,0,0,0,0,0,0),0,0,(0,0,0,1))
jump_data=[10,10,400]
x5.jump(handle, pos, jump_data)
x5.wait_move_done(handle)
# 沿笛卡尔空间 jump 门型运动到笛卡尔目标点(带附加动作)
pos = x5.Point((240,200,0,0,0,0,0,0,0),0,0,(0,0,0,1))
Data = (10,0,0,0,0,0,0,0,0)
add_data = x5.MovPointAdd(vel=50, acc=50, offset=-1, offset_data = Data)
jump_data=[10,10,-10]
x5.jump(handle, pos, jump_data, add_data)
x5.wait_move_done(handle)

```

jogl_rel | 沿笛卡尔坐标系方向增量运动

在笛卡尔坐标系下，将末端沿指定方向进行增量运动

原型

```

def jogl_rel(handle: int, frame_direction: int, delta: float, type: int = 1, rtcp: int = 0) -> None:

```

参数

- `handle (int)`: 机器人句柄号
- `frame_direction (int)`: 轴号(0:X,1:Y,2:Z,3:A,4:B,5:C,6:E1,7:E2,8:E3)
- `delta (float)`: 增量值(mm)
- `type (int)`: 坐标系类型(1: 世界坐标系,2: 工具坐标系,3: 用户坐标系)
- `rtcp (int)`: 外部工具坐标系使用标识位(0:不使用 1:使用)

执行条件

- 机器人上使能，手动模式

示例

```

import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 沿关节运动到笛卡尔目标点

```

```
x5.movj(handle,point=x5.Point((200,200,0,0,0,0,0,0,0),0,0,(0,0,0,1)))
x5.wait_move_done(handle)
# 将末端沿笛卡尔方向进行增量运动
index, delta, type, rtcp = 0, -10, 1, 0
x5.jogl_rel(handle, index, delta, type, rtcp)
x5.wait_move_done(handle)
```

💡 备注

如果上一条运动未结束时，下发下一条运动，会从下发时机器人位置开始计算。

jogl | 沿笛卡尔坐标系方向持续运动

在笛卡尔坐标系下，将机器人末端或外部轴沿指定方向持续运动

原型

```
def jogl(handle: int, frame_direction: int, direction: int, type: int = 1, rtcp: int = 0) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `frame_direction (int)`: 轴号(0:X,1:Y,2:Z,3:A,4:B,5:C,6:E1,7:E2,8:E3)
- `direction (int)`: 运动方向(0:正方向,1:负方向)
- `type (int)`: 坐标系类型(1: 世界坐标系, 2: 工具坐标系, 3: 用户坐标系)
- `rtcp (int)`: 外部工具坐标系使用标识位(0:不使用 1:使用)

执行条件

- 机器人上使能，手动模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 沿关节运动到笛卡尔目标点
x5.movj(handle,point=x5.Point((200,200,0,0,0,0,0,0,0),0,0,(0,0,0,1)))
```

```
x5.wait_move_done(handle)
# 将末端沿笛卡尔方向进行持续运动
index, dir, type, rtcp = 0, 0, 1, 0
x5.jogl(handle, index, dir, type, rtcp)
x5.wait_move_done(handle)
```

💡 备注

如果上一条运动未结束时，下发下一条运动，会从下发时机器人位置开始计算。

jogj_rel | 沿关节坐标系方向增量运动

在关节坐标系下,将末端沿指定方向进行增量运动

原型

```
def jogj_rel(handle: int, frame_direction: int, delta: float, rtcp: int = 0):
```

参数

- `handle (int)`: 机器人句柄号
- `frame_direction (int)`: 坐标系方向(0:J1,1:J2,2:J3,3:J4,4:J5,5:J6,6:E1,7:E2,8:E3)
- `delta (float)`: 增量值(°)
- `rtcp (int)`: 外部工具坐标系使用标识位(0:不使用 1:使用)

执行条件

- 机器人上使能，手动模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 沿关节运动到笛卡尔目标点
x5.movj(handle, point=x5.Point((200,200,0,0,0,0,0,0),0,0,(0,0,0,1)))
x5.wait_move_done(handle)
# 将末端沿关节方向进行增量运动
index, delta, rtcp = 0, -10, 0
```

```
x5.jogj_rel(handle, index, delta, rtcp)
x5.wait_move_done(handle)
```

💡 备注

如果上一条运动未结束时，下发下一条运动，会从下发时机器人位置开始计算。

jogj | 沿关节坐标系方向持续运动

在关节坐标系下,将末端沿指定方向进行持续运动

原型

```
def jogj(handle: int, frame_direction: int, direction: int, rtcp: int = 0):
```

参数

- `handle (int)`: 机器人句柄号
- `frame_direction (int)`: 坐标系方向(0:J1,1:J2,2:J3,3:J4,4:J5,5:J6,6:E1,7:E2,8:E3)
- `direction (int)`: 运动方向(0:正方向,1:负方向)
- `rtcp (int)`: 外部工具坐标系使用标识位(0:不使用 1:使用)

执行条件

- 机器人上使能，手动模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 沿关节运动到笛卡尔目标点
x5.movj(handle, point=x5.Point((200,200,0,0,0,0,0,0),0,0,(0,0,0,1)))
x5.wait_move_done(handle)
# 将末端沿关节方向进行持续运动
index, dir, type, rtcp = 0, 0, 1, 0
x5.jogj(handle, index, dir, rtcp)
x5.wait_move_done(handle)
```

 备注

如果上一条运动未结束时，下发下一条运动，会从下发时机器人位置开始计算。

movsj | 关节坐标系下单轴运动

关节坐标系下，将单个关节运动到目标位置

原型

```
def movsj(handle: int, joint: int, data: float) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `joint (int)`: 关节序号(0:J1,1:J2,2:J3,3:J4,4:J5,5:J6,6:E1,7:E2,8:E3)
- `data (float)`: 目标位置

执行条件

- 机器人上使能，手动或自动命令模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: \n{handle}\n")
# 单个关节轴运动到指定位置
index, data = 0, 10
x5.movsj(handle, index, data)
x5.wait_move_done(handle)
index, data = 0, 0
x5.movsj(handle, index, data)
x5.wait_move_done(handle)
# 如果是七轴协作机器人，J7 目标值为 30，则定义如下
index, data = 6, 30
x5.movsj(handle, index, data)
x5.wait_move_done(handle)
```

servoj | 在线实时控制关节位置

在线实时控制关节位置

函数原型：

```
def servoj(handle, joint, cmdt, lookahead, gain, vel, acc) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `joint (Joint)`: 目标关节位置值，具体内容请参考 [Joint](#) 结构体
- `cmdt (double)`: 指令最小执行时间，范围 0.0 ~ 2.0，单位[s]
- `lookahead (double)`: 暂无使用
- `gain (double)`: 跟随目标位置的比例增益参数，范围 0.0 ~ 50.0，值越大机器人反应越快，但可能导致不稳定振动。
- `vel (double)`: 速度，范围 0.0 ~ 100.0，单位[%]
- `acc (double)`: 加速度，范围 0.0 ~ 100.0，单位[%]

执行条件

- 机器人上使能，手动或自动命令模式

调用示例：

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: \n{handle}\n")
# 运动到 j1=10,j2=10,j3=0,j4=10,j5=0,j6=0 点位，0.032 秒内执行完成。
jpos = x5.Joint(10, 10, 0, 10, 0, 0, 0, 0, 0)
x5.servoj(handle, jpos, 0.032, 0, 5, 100, 100)
```

备注

使用 servoj 与 servol 接口前，请联系厂家详细了解情况。

servol | 在线实时控制 TCP 位置

在线实时控制 TCP 位置

函数原型：

```
def servol(handle, point, cmdt, lookahead, gain, vel, acc) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `point (Point)`: 目标笛卡尔点位，具体内容请参考 [Point](#) 结构体
- `cmdt (double)`: 指令最小执行时间，范围 0.0 ~ 2.0，单位[s]
- `lookahead (double)`: 暂无使用
- `gain (double)`: 跟随目标位置的比例增益参数，范围 0.0 ~ 50.0，值越大机器人反应越快，但可能导致不稳定振动。
- `vel (double)`: 速度，范围 0.0 ~ 100.0，单位[%]
- `acc (double)`: 加速度，范围 0.0 ~ 100.0，单位[%]

执行条件

- 机器人上使能，手动或自动命令模式

调用示例：

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: \n{handle}\n")
# 运动到 x=200,y=200,z=-10,a=0,b=0,c=30 点位，0.032 秒内执行完成。
pos = x5.Point((200,200,-10,0,0,30,0,0,0),0,0,(0,0,0,1))
x5.servol(handle, pos, 0.032, 0, 5, 100, 100)
```

备注

使用 `servoj` 与 `servol` 接口前，请联系厂家详细了解情况。

实时引导运动 RGM

实时引导运动 Realtime-Guided-Motion（简称：RGM）功能为高级用户设计的，它通过绕过相关路径规划的方式为机器人控制器提供了一个实时引导运动的接口，用户可用来高速读取运动系统的位置和向运动系统高速写入位置（最高控制周期可达 2ms）。

- 由于机器人控制器中的 RGM 绕过了路径规划，因此用户输入创建的机器人路径，需确保发送给控制器的位置引导尽量平滑。
- 为保证实时性，客户端环境要求使用搭配具备实时处理能力的操作系统（如：Ubuntu-Linux），否则会产生机器人抖动现象。[详见附录：Ubuntu-Linux 实时环境配置](#)

备注

使用实时引导运动 RGM 功能前，请联系厂家详细了解情况。

使用 RGM 功能时需注意：

- 1.当使用 [rgm_start](#) 启动 RGM 模式后，以下情况会自动退出：
 - 客户端连接断开；
 - 系统报警
 - 系统下使能；
 - 退出自动命令模式；
 - 执行 stop、abort、rgm_stop 命令；
- 2.RGM 模式下执行 [rgm_servoj](#)、[rgm_servol](#) 命令时，仍可使用 [pause](#)、[resume](#) 命令暂停或继续运动。
- 3.RGM 模式下无法执行 [movl](#)、[movj](#) 等运动驱动命令。

rgm_init | RGM 初始化

RGM 参数初始化。

函数原型：

```
def rgm_init(handle, port, cycle_rate, userfunc) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `port (int)`: 端口号, 范围 50200 ~ 50209
- `cycle_rate (int)`: 控制周期 $2\text{ms} \times \text{cycle_rate}$, 范围 1 ~ 10, 最小控制周期 2ms
- `userfunc`: 用户回调函数

相关接口

[rgm_destroy](#)、[rgm_start](#)、[rgm_stop](#)、[rgm_servoj](#)、[rgm_servol](#)

示例：

详见 [RGM 示例](#)

rgm_destroy | RGM 销毁

RGM 销毁, 退出程序或再次使用 `rgm_init` 前, 建议使用本接口回收资源。

函数原型：

```
def rgm_destroy(handle) -> None:
```

参数

- `handle (int)`: 机器人句柄号

相关接口

[rgm_init](#)、[rgm_start](#)、[rgm_stop](#)、[rgm_servoj](#)、[rgm_servol](#)

示例：

详见 [RGM 示例](#)

rgm_start | RGM 启动

启动 RGM 功能, 系统进入到 RGM 模式, 并建立通讯连接。通过 [SystemState](#) 获取当前系统是否处于 RGM 模式。

函数原型：

```
def rgm_start(handle) -> None:
```

参数

- `handle (int)`: 机器人句柄号

执行条件

机器人上使能，自动命令模式。

相关接口

[rgm_init](#)、[rgm_stop](#)、[rgm_servoj](#)、[rgm_servol](#)

示例：

详见 [RGM 示例](#)

rgm_stop | RGM 停止

停止 RGM 功能，系统退出 RGM 模式，并断开通讯连接。

函数原型：

```
def rgm_stop(handle) -> None:
```

参数

- `handle (int)`: 机器人句柄号

执行条件

机器人上使能，自动命令模式。

示例：

详见 [RGM 示例](#)

相关接口

[rgm_start](#)、[rgm_servoj](#)、[rgm_servol](#)

示例：

详见 [RGM 示例](#)

rgm_servoj | RGM 实时关节位置运动

RGM 模式下，实时引导控制关节位置运动。

函数原型：

```
def rgm_servoj(handle, joint, cmdt, lookahead, gain, vel, acc) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `joint (Joint)`: 目标笛卡尔点位，具体内容请参考 [Joint](#) 结构体
- `cmdt (double)`: 指令最小执行时间，范围 0.0 ~ 2.0，单位[s]
- `lookahead (double)`: 暂无使用
- `gain (double)`: 跟随目标位置的比例增益参数，范围 0.0 ~ 50.0，值越大机器人反应越快，但可能导致不稳定振动。
- `vel (double)`: 速度，范围 0.0 ~ 100.0，单位[%]
- `acc (double)`: 加速度，范围 0.0 ~ 100.0，单位[%]

相关接口

[rgm_start](#)、[rgm_stop](#)、[rgm_servol](#)

执行条件

机器人上使能，RGM 模式。

示例：

详见 [RGM 示例](#)

rgm_servol | RGM 实时 TCP 位置运动

RGM 模式下，实时引导控制 TCP 位置运动。

函数原型：

```
def rgm_servol(handle, point, cmdt, lookahead, gain, vel, acc) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `joint (Point)`: 目标笛卡尔点位，具体内容请参考 [Point](#) 结构体
- `cmdt (double)`: 指令最小执行时间，范围 0.0 ~ 2.0，单位[s]
- `lookahead (double)`: 暂无使用
- `gain (double)`: 跟随目标位置的比例增益参数，范围 0.0 ~ 50.0，值越大机器人反应越快，但可能导致不稳定振动。
- `vel (double)`: 速度，范围 0.0 ~ 100.0，单位[%]
- `acc (double)`: 加速度，范围 0.0 ~ 100.0，单位[%]

相关接口

[rgm_start](#)、[rgm_stop](#)、[rgm_servoj](#)

执行条件

机器人上使能，RGM 模式。

示例：

详见 [RGM 示例](#)

RGM 示例

```
import xapi.api_dev as x5
from xapi.api.rgm import *
import threading
import time

static_forward_flag = 1
static_once = True # 执行一次

@rgm_callback
def userfunc(handle, state: RGMRobState) -> int:

    global static_forward_flag, static_once, static_pos
    if static_once:
        static_forward_flag = 1
        static_once = False

    joint = state.joint
    # 修改位置逻辑
    if static_forward_flag == 1:
        joint.j1 = joint.j1+1
        joint.j2 = joint.j2+1
        joint.j4 = joint.j4+1
        if(joint.j1 > -20):
            static_forward_flag = 0
    else:
        joint.j1 = joint.j1-1
        joint.j2 = joint.j2-1
        joint.j4 = joint.j4-1
        if(joint.j1 < 20):
            static_forward_flag = 1
```

```

x5.rgm_servoj(handle, joint, cmdt=0, lookahead=0, gain=50, vel=100, acc=100)
# 或者执行 rgm_servol
# point = x5.cnvrt_p(handle, joint, 0, 0)
# x5.rgm_servol(handle, point, cmdt=0, lookahead=0, gain=50, vel=100, acc=100)
return 0

if __name__ == "__main__":
    global static_joint
    x5.enable_debug_output(True)
    # 创建 TCP 连接
    handle = x5.connect("168.168.10.42")

    # 初始化 RGM 资源
    x5.rgm_init(handle, 50201, 1, userfunc)
    x5.reset(handle)
    # 切换到自动 CMD 模式
    x5.set_system_mode(handle, x5.SYSTEM_MODE_AUTOCMD)
    # 上使能
    x5.enable_servo(handle, 1)
    time.sleep(1)

    # 运动到起始位置
    static_joint = x5.Joint(0,0,0,0,0,0,0,0,0)
    x5.movj(handle, static_joint)
    x5.wait_move_done(handle)

    # 启动 RGM 模式
    x5.rgm_start(handle)

    try:
        time.sleep(30) # 延迟 30 秒
    except KeyboardInterrupt:
        pass
    finally:
        # 停止 RGM 功能
        x5.rgm_stop(handle)
        # 释放 RGM 资源
        x5.rgm_destroy(handle)
        # 断开 TCP 连接
        x5.disconnect(handle)
        # 析构 API 日志
        x5.logger_shutdown()

```

运动配置

set_speed | 设置机器人全局速度

设置机器人的全局运动速度百分比

原型

```
def set_speed(handle, scale) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `scale (int)`: 全局速度百分比(1~100), 单位: %

执行条件

- 任意模式

示例:

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 设置全局速度为 66%
x5.set_speed(handle, 66)
```

get_speed | 获取机器人全局速度

获取机器人的全局运动速度百分比

原型

```
def get_speed(handle) -> int:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `scale (int)`: 全局速度比例(1~100)，单位：%

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 获取全局速度
scale = x5.get_speed(handle)
print(f"速度: {scale}\n")
```

get_cur_move_state | 获取当前运动指令完成状态

获取当前运动指令完成状态

原型

```
def get_cur_move_state(handle) -> int:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `state (int)`: 机器人当前运动指令完成状态 (0:未完成, 1:已完成, 2:未找到)

⚠ 警告

当运动过程中, 执行 stop 或触发机器人报警, 将导致运动清空, 请重新调用运动接口。

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
jp0 = x5.Joint(0,0,0,0,0,0)
jp1 = x5.Joint(30,0,0,0,0,0)
# 设置全局速度为 10%
x5.set_speed(handle, 10)
# 运动到 jp0, 然后运动到 jp1
x5.movj(handle, jp0)
x5.movj(handle, jp1)
# 当前状态应该为运动到 jp1 点运动未完成
assert x5.get_cur_move_state(handle) == 0
x5.wait_move_done(handle)
# 当前状态应该为运动到 jp1 点运动完成
assert x5.get_cur_move_state(handle) == 1
x5.stop(handle)
x5.wait_move_done(handle)
```

```
# 当前状态应该为运动到 jp1 点运动未找到  
assert x5.get_cur_move_state(handle) == 2
```

wait_move_done | 等待机器人运动完成

等待机器人当前的运动命令全部执行完成

原型

```
def wait_move_done(handle: int, timeout_ms: int = -1) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `timeout_ms (int, optional)`: 超时时间(ms), 默认-1 表示无限等待。

⚠ 警告

如设置超时时间, 当等待时间大于超时时间, 则接口报错, 而机器人继续当前运动。

执行条件

- 手动或自动命令模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 执行条件
try:
    # 沿笛卡尔空间运动到目标点
    x5.movj(handle, point=x5.Joint())
    # 等待运动执行完成
    x5.wait_move_done(handle)
    t.sleep(1)
    # 沿笛卡尔空间运动到目标点
    x5.movj(handle, point=x5.Point((200, 200, 0, 0, 0, 0, 0, 0, 0), 0, 0, (0, 0, 0, 1)))
    # 等待运动执行完成
    x5.wait_move_done(handle)
except x5.RobException as rex:
    print(f"报错接口: {rex.api_name}")
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
t.sleep(1)
```

程序管理

前台程序

run | 运行程序

运行主程序或指定程序

原型

```
def run(handle, program_name = None, start_line = 1) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `program_name (str, optional)`: 目标程序名称（不带后缀）,默认空时从主程序开始运行
- `start_line (int, optional)`: 程序启动行号(首行为 1),默认从首行 1 开始

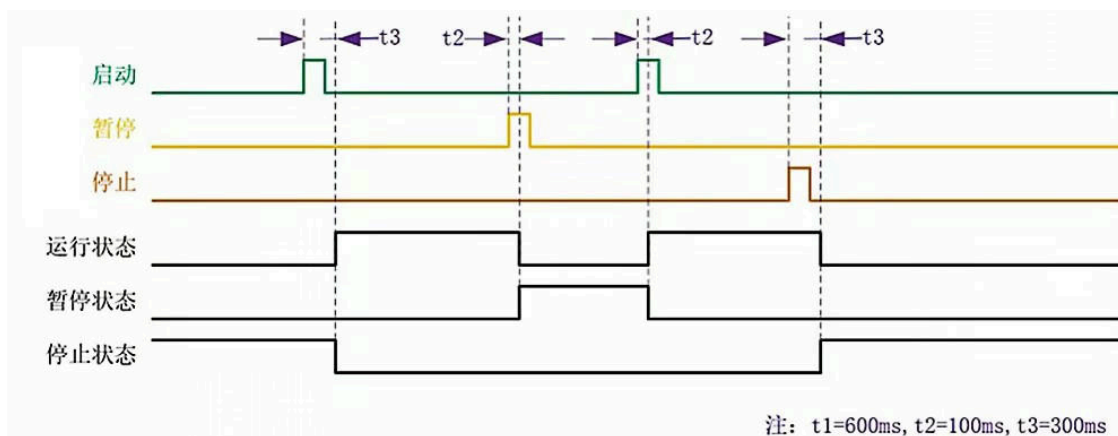
⚠ 警告

如指定程序起始行号运行程序，起始行号必须在实际程序行号范围内，否则机器人报警。

执行条件

- 机器人上使能，自动模式

时序图：



示例

```
# 从主程序首行开始自动运行
x5.run(handle)
# 从指定程序首行开始运行程序 test.lua 或 test.xml(需为机器人中已完成编译的程序名称)
```

```
x5.run(handle,"test")
# 从指定程序、指定行开始自动运行
x5.run(handle,"test", 3)
```

备注

1. 接口仅支持机器人上使能、自动模式下执行。
2. program_name 需为机器人中已存在且完成编译的程序文件名称。
3. 运行、暂停、恢复运行、停止程序的时序，请参考如上时序图。

pause | 暂停程序或运动

暂停运行当前程序或运动

原型

```
def pause(handle) -> None:
```

参数

- handle (int): 机器人句柄号

执行条件

- 机器人上使能，自动或者自动命令模式

resume | 恢复程序或运动

恢复运行当前程序或运动

原型

```
def resume(handle) -> None:
```

参数

- handle (int): 机器人句柄号

执行条件

- 机器人上使能，自动或者自动命令模式

备注

1. 接口用于自动模式下，在暂停程序后，恢复机器人程序运行。
2. 接口用于自动命令模式下，暂停机器人运动后，恢复机器人运动。
3. 接口仅能恢复暂停机器人 movl、movj、movc、jump、jumpl、movsj 运动。
4. 恢复运行时序详见暂停运行接口时序图。

stop | 停止程序或运动

停止运行当前程序或运动

原型

```
def stop(handle) -> None:
```

参数

- `handle (int)`: 机器人句柄号

执行条件

- 机器人上使能，手动、自动或者自动命令模式

备注

1. 自动模式下，接口用于停止机器人程序，之后可通过 start 接口进行停止再启动。
2. 手动或自动 CMD 模式下，接口用于停止机器人运动，并清空队列，无法继续运动。
3. 调用 stop 和 abort 接口停止机器人运动时，机器人会需要时间来进行减速停止。
4. 如有单独线程在持续下发运动，调用 stop 接口后，请自行处理停止线程下发。
5. 当运行后台非静态程序时，使用 stop 接口，将停止后台运行，建议使用静态任务。
6. 停止状态时序详见暂停运行接口时序图。

set_path_back_type | 设置停止再启动返回方式

设置停止再启动时是否返回原停止位置。

原型

```
def set_path_back_type(handle, type = 0) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `type (int)`: 返回方式: 0-不返回停止位置, 1-返回停止位置

执行条件

- 机器人自动模式

示例

```
# 设置停止再启动时不返回原停止位置。
x5.set_path_back_type(handle)
# 设置停止再启动时返回原停止位置。
x5.set_path_back_type(handle, 1)
```

备注

搭配 start 接口使用，可实现停止再启动功能

start | 开始运行程序

停止再启动，开始运行程序

原型

```
def start(handle) -> None:
```

参数

- `handle (int)`: 机器人句柄号

执行条件

- 机器人自动模式

备注

可使用 set_path_back_type 接口，设置当程序停止再启动时是否返回原停止位置(未使用 set_path_back_type 接口设置返回方式, 则内部默认选择[返回之前停止位置])。

前台程序示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
```

```

# 执行条件
try:
    x5.abort(handle)
    # 退出远程模式
    x5.set_remote(handle, 0)
    t.sleep(1)
    # 切换自动模式
    x5.set_system_mode(handle, x5.SYSTEM_MODE_AUTO)
    t.sleep(1)
    # 系统上使能
    x5.enable_servo(handle, 1)
    t.sleep(1)
    # 获取当前系统状态
    local_state = x5.get_system_state(handle)
    # 判断是否为自动模式，伺服上电，机器人停止状态
    if local_state.mode == 2 and local_state.enable == 1 and local_state.run == 3:
        # 自动运行指定程序 test.lua 或 test.xml(需为机器人中已完成编译的程序名称)
        x5.run(handle, "test")
    t.sleep(1)
    # 获取当前系统状态
    local_state = x5.get_system_state(handle)
    # 暂停运行当前程序，判断是否为自动模式，伺服上电，机器人运行状态
    if local_state.mode == 2 and local_state.enable == 1 and local_state.run == 1:
        x5.pause(handle)
    t.sleep(1)
    # 获取当前系统状态
    local_state = x5.get_system_state(handle)
    # 恢复运行当前程序，判断是否为自动模式，伺服上电，机器人暂停状态
    if local_state.mode == 2 and local_state.enable == 1 and local_state.run == 2:
        x5.resume(handle)
    t.sleep(1)
    # 获取当前系统状态
    local_state = x5.get_system_state(handle)
    # 停止当前运行程序，判断是否为自动模式，伺服上电，机器人运行状态
    if local_state.mode == 2 and local_state.enable == 1 and local_state.run == 1:
        x5.stop(handle)
        # 等待机器人减速停止运动完成
        x5.wait_move_done(handle)
    t.sleep(1)
    # 切换手动模式
    x5.set_system_mode(handle, x5.SYSTEM_MODE_MANUAL)
    t.sleep(1)
    # 系统上使能

```

```

x5.enable_servo(handle,1)
t.sleep(1)
x5.jogj_rel(handle, 0, 15)
x5.wait_move_done(handle)
# 切换自动模式
x5.set_system_mode(handle, x5.SYSTEM_MODE_AUTO)
t.sleep(1)
# 设置停止再启动时是否返回原停止位置：是。
x5.set_path_back_type(handle, 1)
t.sleep(1)
# 系统上使能
x5.enable_servo(handle,1)
t.sleep(1)
# 获取当前系统状态
local_state = x5.get_system_state(handle)
# 判断是否为自动模式，伺服上电，机器人停止状态
if local_state.mode == 2 and local_state.enable == 1 and local_state.run == 3:
    # 停止再启动，继续运行前台程序
    x5.start(handle)
except x5.RobException as rex:
    print(f"报错接口: {rex.api_name}")
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
t.sleep(1)

```

后台程序

备注

可在任意模式下，通过 API 接口对静态、动态、指令后台进行控制。

daemon_run | 运行或恢复后台程序

运行或恢复后台程序

原型

```
def daemon_run(handle, name) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `name (str)`: 后台程序名, 如 “test” (无论是.lua 或.xml 程序)

daemon_pause | 暂停后台程序

暂停后台程序

原型

```
def daemon_pause(handle, name) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `name (str)`: 后台程序名, 同 daemon_run 参数

daemon_stop | 停止后台程序

停止后台程序

原型

```
def daemon_stop(handle, name) -> None:
```

参数

- `handle (int)`: 机器人句柄号。
- `name (str)`: 后台程序名, 同 daemon_run 参数, 若不传入, 则默认停止所有任务。

后台程序示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 执行条件
try:
    # 运行后台程序 test_dae.lua 或者 test_dae.xml
    x5.daemon_run(handle, "test_dae")
    t.sleep(1)
    # 暂停后台程序 test_dae.lua 或者 test_dae.xml
    x5.daemon_pause(handle, "test_dae")
    t.sleep(1)
    # 恢复后台程序 test_dae.lua 或者 test_dae.xml
    x5.daemon_run(handle, "test_dae")
    t.sleep(1)
    # 停止后台程序 test_dae.lua 或者 test_dae.xml
    x5.daemon_stop(handle, "test_dae")
    t.sleep(1)
except x5.RobException as rex:
    print(f"报错接口: {rex.api_name}")
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
t.sleep(1)
```

程序模块

备注

建议在手动模式下，使用程序模块等相关 API 接口。

get_module_name_list | 获取模块名列表

获取模块名列表

原型

```
def get_module_name_list(handle) -> list[str]:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `module_name_list (list[str])`: 模块名列表

load_module | 加载模块

加载模块

原型

```
def load_module(handle, module_name) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `module_name (str)`: 模块名, 如 “test”

执行条件

- 手动模式，且所加载模块未被占用

get_compile_state | 获取系统程序编译状态

获取系统程序编译状态

原型

```
def get_compile_state(handle) -> int:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `state (int)`: 系统编译状态 (0:未编译, 1:编译中, 2:已编译) 编译失败会置为 0

get_program_info | 获取当前模块的程序信息

获取当前模块的程序信息

原型

```
def get_program_info(handle) -> List[ProgramInfo]:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `programs (List[ProgramInfo])`: 程序信息列表, 详见 [ProgramInfo](#) 数据类型

程序模块示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 执行条件
try:
    # 获取模块名列表
    result = x5.get_module_name_list(handle)
    print(f"\n 模块名列表: {result}\n")
    t.sleep(1)
    # 加载模块名列表中获取到的模块 test.Mod
    x5.load_module(handle, "test")
    t.sleep(1)
    # 获取系统程序编译状态
    result = x5.get_compile_state(handle)
    print(f"\n 系统程序编译状态: {result}\n")
    t.sleep(1)
```

```
# 获取当前模块的程序信息
result = x5.get_program_info(handle)
print(f"\n 当前模块的程序信息: {result[0]}\n")
t.sleep(1)
except x5.RobException as rex:
    print(f"报错接口: {rex.api_name}")
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
```

IO 管理

IO 设定

set_do | 设置数字量输出状态

设置指定端口的数字量输出（DO）状态。

原型

```
def set_do(handle, index, state) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: DO 端口号(0~247)
- `state (int)`: DO 状态(0:关闭,1:打开)

执行条件

- 手动模式、自动命令模式

示例

```
import xapi.api as x5

# 连接机器人
ip = "192.168.1.6"
handle = x5.connect(ip)

# 设置 DO 信号
index, data = 0, 1
x5.set_do(handle, index, data)
x5.wait_set_do_done(handle)
# 查看 DO 信号是否被成功修改
actual_data = x5.get_do(handle, index)
print(f"数字输出 D00 = {actual_data}")
# 输出:
# 数字输出 D00 = 1
```

get_do | 获取数字量输出状态

获取一定数量端口的数字量输出（DO）状态。如果不指明端口号，则获取所有。

原型

```
def get_do(handle, index=None, count=None) -> Union[int, List[int]]:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int, optional)`: DO 端口号(0~247)。如果不指明端口号，则获取所有数字输出量。
- `count (int, optional)`: 要获取的 DO 数量，仅当 `index` 存在时可指定，默认为 1

返回值

- `Union[int, List[int]]`: 单个 DO 状态或 DO 状态列表(1: 打开,0: 关闭)

执行条件

- 任意模式

示例

```
import xapi.api as x5

# 连接机器人
ip = "192.168.1.6"
handle = x5.connect(ip)

# 获取指定端口的数字输出
index=0
data = x5.get_do(handle, index)
print(f"D00 端口状态: {data}")

# 获取端口 D00-D09 的数字输出
index, count = 0, 10
data = x5.get_do(handle, index, count)

# 获取所有端口的数字输出
data = x5.get_do(handle)
for index in range(0, 248):
    print(f"D0{index}: {data[index]}")
```

set_ao | 设置模拟量输出电压值

设置指定端口的模拟量输出（AO）电压值。

原型

```
def set_ao(handle, index, value) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: AO 端口号(0~15)
- `value (int)`: AO 端口的电压值，单位：mV 毫伏。数值不能超过“IO 设定”模块的取值范围

执行条件

- 手动模式、自动命令模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 设置 AO 信号
index, value= 0, 111
x5.set_ao(handle, index, value)
t.sleep(0.05)
```

get_ao | 获取模拟量输出电压值

获取指定端口的模拟量输出（AO）电压值。如果不指明端口号，则获取所有。

原型

```
def get_ao(handle, index=None, count=None) -> Union[int, List[int]]:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int, optional)`: AO 端口号(0~15)。如果不指明端口号，则获取所有模拟输出。
- `count (int, optional)`: 要获取的 AO 数量，仅当 `index` 存在时可指定，默认为 1

返回值

- `Union[int, List[int]]`: 单个 AO 值或 AO 列表（单位：mV）

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 获取指定模拟量输出端口状态
index=0
data = x5.get_ao(handle, index)
print(f"A00 端口状态: {data}")
# 获取端口 A00-A09 的模拟量输出
index, count = 0, 10
data = x5.get_ao(handle, index, count)
# 获取所有模拟量输出端口状态
data = x5.get_ao(handle)
print(f"A0 端口状态: {data[0]}\n")
```

get_di | 获取数字量输入状态

获取指定端口的数字量输入（DI）状态。如果不指明端口号，则获取所有。

原型

```
def get_di(handle, index=None, count: Optional[int] = None) -> Union[int, List[int]]:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int, optional)`: DI 端口号(0~247)如果不指明端口号，则获取所有 DI。
- `count (int, optional)`: 要获取的 DI 数量，仅当 `index` 存在时可指定，默认为 1

返回值

- `Union[int, List[int]]`: 单个 DI 状态或 DI 状态列表(1: 打开,0: 关闭)

执行条件

- 任意模式

示例

```
import xapi.api as x5

# 连接机器人
ip = "192.168.1.6"
handle = x5.connect(ip)

# 获取指定端口的数字输入
index=0
data = x5.get_di(handle, index)
print(f"DI0 端口状态: {data}")

# 获取端口 DI0-DI9 的模拟量输出
index, count = 0, 10
data = x5.get_di(handle, index, count)

# 获取所有端口的数字输入
data = x5.get_di(handle)
for index in range(0, 248):
    print(f"DI{index}: {data[index]}")
```

get_ai | 获取模拟量输入电压值

获取一定数量端口的模拟量输入（AI）电压值。如果不指明端口号，则获取所有。

原型

```
def get_ai(handle, index=None, count=None) -> Union[int, List[int]]:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int, optional)`: AI 端口号(0~15)。如不指明端口号，则获取所有模拟输出。
- `count (int, optional)`: 要获取的 AI 数量，仅当 `index` 存在时可指定，默认为 1

返回值

- `Union[int, List[int]]`: 单个 AI 值或 AI 列表（单位：mV）

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 获取指定模拟量输入端口状态
index=0
data = x5.get_ai(handle, index)
print(f"AI0 端口状态: {data}")
# 获取端口 AI0-AI9 的模拟量输出
index, count = 0, 10
data = x5.get_ai(handle, index, count)
# 获取所有模拟量输入端口状态
data = x5.get_ai(handle)
print(f"AI0 端口状态: {data[0]}\n")
```

get_ei | 获取编码器输入值

获取一定数量端口的编码器编码器输入（EI）值。如果不指明端口号，则获取所有。

原型

```
def get_ei(handle, index=None, count=None) -> Union[int, List[int]]:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int, optional)`: EI 端口号(0~7)。如果不指明端口号，则获取编码器输入。
- `count (int, optional)`: 要获取的 EI 数量，仅当 `index` 存在时可指定，默认为 1

返回值

- `Union[int, List[int]]`: 单个 EI 值或 EI 列表

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 获取指定编码器输入端口编码器值
index=0
data = x5.get_ei(handle, index)
print(f"EI0 端口编码器值: {data}")

# 获取端口 EI0-EI2 的模拟量输出
index, count = 0, 3
data = x5.get_ei(handle, index, count)

# 获取所有编码器输入端口编码器值
data = x5.get_ei(handle)
print(f"EI0 端口编码器值: {data[0]}\n")
```

IO 强制

set_force_di | 强制设置数字量输入状态

强制设置数字量输入（DI）的状态。如果不输入 data，则将强制状态复位。

原型

```
def set_force_di(handle, index, value=None) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: DI 端口号(0~247)
- `value (int, optional)`: DI 强制状态值(0: 强制打开, 1: 强制关闭)。如果不输入数值, 则将强制状态复位

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 强制 DI 信号
index, value= 0, 1
x5.set_force_di(handle, index, value)
t.sleep(0.05)
```

get_force_di | 获取数字量输入强制状态

获取指定数字量输入（DI）的强制状态。如果不指明端口号，则获取所有。

原型

```
def get_force_di(handle, index=None, count=None) -> Union[int, List[int]]:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int, optional)`: DI 端口号(0~247)。如果不指明端口号，则获取所有 DI 强制状态。
- `count (int, optional)`: 要获取的 DI 数量，仅当 `index` 存在时可指定，默认为 1

返回值

- `Union[int, List[int]]`: 单个强制 DI 状态或强制 DI 状态列表(1: 已强制,0: 未强制)

执行条件

- 任意模式

示例

```
import xapi.api as x5

# 连接机器人
ip = "192.168.1.6"
handle = x5.connect(ip)

# 获取指定 DI 端口的强制状态
index=0
data = x5.get_force_di(handle, index)
print(f"DI0 端口的强制状态: {data}")

# 获取端口 DI0-DI9 的强制状态
index, count = 0, 10
data = x5.get_force_di(handle, index, count)

# 获取所有 DI 端口的强制状态
data = x5.get_force_di(handle)
for index in range(0, 248):
    print(f"DI{index}强制状态: {data[index]}")
```

reset_force_di | 复位数字量输入强制状态

复位数字量输入（DI）的强制状态。

原型

```
def reset_force_di(handle: int, index: int) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: DI 端口号(0~247)

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 强制 DI 信号
index, value= 0, 1
x5.set_force_di(handle, index, value)
t.sleep(0.05)
# 复位 DI 强制信号
x5.reset_force_di(handle, index)
```

set_force_ai | 强制设置模拟量输入电压值

强制设置模拟量输入（AI）的电压值。如果不输入 value，则将强制状态复位。

原型

```
def set_force_ai(handle, index, value = None) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: AI 端口号(0~15)
- `value (int, optional)`: AI 强制状态值, 单位: mv 毫伏。如不输入数值, 则将强制状态复位。

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 强制 AI 信号
index, data= 0, 77
x5.set_force_ai(handle, index, data)
t.sleep(0.05)
# 复位 AI 强制信号
x5.set_force_ai(handle, index)
```

get_force_ai | 获取模拟量输入强制状态

获取一定数量模拟量输入（AI）的强制状态。如果不指明端口号，则获取所有。

原型

```
def get_force_ai(handle, index = None, count = None) -> Union[int, List[int]]:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int, optional)`: AI 端口号(0~15)，不传时获取所有端口，传入时与 `count` 组合使用。
- `count (int, optional)`: 要获取的 AI 数量，仅当 `index` 存在时可指定，默认为 1

返回值

- `Union[int, List[int]]`: 单个强制 AI 状态或强制 AI 状态列表(1: 已强制,0: 未强制)

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 获取指定模拟量输入端口状态
index=0
data = x5.get_force_ai(handle, index)
print(f"AI0 端口强制状态: {data}")
# 获取模拟量输入端口 AI0-AI9 的强制状态
index, count = 0, 10
data = x5.get_force_ai(handle, index, count)
# 获取所有模拟量输入端口的强制状态
data = x5.get_force_ai(handle)
print(f"AI0 端口强制状态: {data[0]}\n")
```

reset_force_ai | 复位模拟量输入强制状态

复位模拟量输入（AI）的强制状态。

原型

```
def reset_force_ai(handle, index) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: AI 端口号(0~15)

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 强制 AI 信号
index, data= 0, 100
x5.set_force_ai(handle, index, data)
t.sleep(0.05)
# 复位 AI 强制信号
x5.reset_force_ai(handle, index)
```

reset_all_force_io | 复位所有端口的强制状态

复位所有信号端口的强制状态，包括数字量输入（DI）与模拟量输入（AI）。

原型

```
def reset_all_force_io(handle) -> None:
```

参数

- `handle (int)`: 机器人句柄号

执行条件

- 任意模式

示例

```
import xapi.api as x5
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 复位所有数字量与模拟量端口强制状态
x5.reset_all_force_io(handle)
```

IO 命令

wait_set_do_done | 等待数字量输出状态执行完成

等待设置数字量输出（DO）操作执行完成。

原型

```
def wait_set_do_done(handle, timeout_ms: int = -1) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `timeout_ms (int)`: 超时时间(ms), 默认-1 表示无限等待。

⚠ 警告

如设置超时时间，当等待时间大于超时时间，则接口报错。

执行条件

- 任意模式

示例

```
import xapi.api as x5

# 连接机器人
ip = "192.168.1.6"
handle = x5.connect(ip)

# 设置 DO 信号
index, data = 0, 1
x5.set_do(handle, index, data)
x5.wait_set_do_done(handle)
# 查看 DO 信号是否被成功修改
actual_data = x5.get_do(handle, index)
print(f"数字输出 DO0 = {actual_data}")
# 输出:
# 数字输出 DO0 = 1
```

💡 备注

接口常用于一次性设置大量数字量输出(DO)端口时使用。

变量管理

数值变量

set_hr | 设置 HR 变量

设置指定 HR 变量的数据，或者设置一组偏移的 HR 变量的数据。

原型

```
def set_hr(handle: int, index: int, data: Union[float, List[float]]) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: HR 变量序号(0~999)
- `data (float|list)`: HR 变量数据

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"控制器句柄号: {handle}")

# 设置指定 HR 变量数据
data1, index1 = 90.0, 4
x5.set_hr(handle, index1, data1)
# 设置所有 HR 变量数据
data2 = []
index2 = 0
for i in range(0, 1000):
    data2.append(90.0)
x5.set_hr(handle, index2, data2)
```

get_hr | 获取 HR 变量

获取指定 HR 变量的数据，或者获取一组偏移的 HR 变量的数据。

原型

```
def get_hr(handle, index, count = 1) -> Union[float, List[float]]:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int, optional)`: HR 变量序号 (0~999)，不传时获取所有端口，传入时需与 `count` 组合使用
- `count (int, optional)`: 要获取的 HR 数量 (1~1000)，仅当 `index` 存在时可指定，默认为 1

返回值

- `Union[float, List[float]]`: 单个 HR 值或 HR 列表

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: \n{handle}\n")

# 获取指定 HR 变量数值
index=0
data = x5.get_hr(handle, index)
print(f"HR0 变量数值: {data}")
# 获取所有 HR 变量数值
index, count = 0, 1000
data = x5.get_hr(handle, index, count)
print(f"HR0 变量数值: {data[0]}")
```

set_mbr | 设置 MBR 变量

设置指定序号的 Modbus 变量的数据

原型

```
def set_mbr(handle, index, value) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: MBR 变量序号(0~99)
- `value (int)`: MBR 变量数值

执行条件

- 手动或者自动命令模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 设置指定 MBR 变量数据
index,value = 4,90
x5.set_mbr(handle, index, value)
```

get_mbr | 获取 MBR 变量

获取指定序号的 Modbus 变量的数据

原型

```
def get_mbr(handle, index = None, count = None) -> Union[int, List[int]]:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int, optional)`: MBR 端口号(0~99),不传时获取所有端口,传入时需与 `count` 组合使用
- `count (int, optional)`: 要获取的 MBR 数量,仅当 `index` 存在时可指定,默认为 1

返回值

- `Union[int, List[int]]`: 单个 MBR 值或 MBR 列表

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 获取指定 MBR 变量数值
index=0
data = x5.get_mbr(handle, index)
print(f"MBR0 变量数值: {data}")
# 获取 MBR 变量 MBR0-MBR9 的数值
index, count = 0, 10
data = x5.get_mbr(handle, index, count)
# 获取所有 MBR 变量数值
data = x5.get_mbr(handle)
print(f"MBR0 变量数值: {data[0]}\n")
```

位置变量

set_pr | 设置 PR 变量

设置指定序号的 PR 变量的数据，或者设置一组偏移的 PR 变量的数据。

原型

```
def set_pr(handle, index, points) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: PR 变量序号(0~999)
- `points (Union[Point, List[Point]])`: 单个 PR 值或 PR 列表

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# set_pr
index1, point = 1, x5.Point((46,-1,39,180,0,0,0,0,0),0,0,(0,0,0,1))
x5.set_pr(handle, index1, point)
# 设置所有 PR 变量数据
points = []
index2 = 0
for i in range(0, 1000):
    points.append(x5.Point((46,-1,39,180,0,0,0,0,0),0,0,(0,0,0,1)))
x5.set_pr(handle, index2, points)
```

get_pr | 获取 PR 变量

获取指定序号的 PR 变量的数据，或者获取一组偏移的 PR 变量的数据。

原型

```
def get_pr(handle, index = None, count = None) -> Union[Point, List[Point]]:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int, optional)`: PR 变量序号(0~999)，不传时获取所有端口，传入时需与 `count` 组合使用
- `count (int, optional)`: 要获取的 PR 数量(1~1000)，仅当 `index` 存在时可指定，默认为 1

返回值

- `value (Point | List[Point])`: 单个 PR 值或 PR 列表

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 获取指定 PR 变量值
index=0
point = x5.get_pr(handle, index)
print(f"PR0 变量数值: {point}")
# 获取连续 PR 变量值
index, count = 3, 4
points = x5.get_pr(handle, index, count)
for i in range(0, 4):
    print(f"PR{3+i}变量数值: {points[i]}")
# 获取所有 PR 变量值
points = x5.get_pr(handle)
```

set_jpr | 设置 JPR 变量

设置指定序号的 JPR 变量的数据，或者设置一组偏移的 JPR 变量的数据。

原型

```
def set_jpr(handle, index, joints) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: JPR 变量序号(0~63)
- `joints (Union[Joint, List[Joint]])`: 单个 JPR 值或 JPR 列表

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 设置指定 JPR 变量
index1, joint = 1, x5.Joint(46,-1,39,180,0,0,0,0)
x5.set_jpr(handle, index1, joint)
# 设置所有 JPR 变量数据
joints = []
index2 = 0
for i in range(0, 64):
    joints.append(x5.Joint(46,-1,39,180,0,0,0,0))
x5.set_jpr(handle, index2, joints)
```

get_jpr | 获取 JPR 变量

获取指定序号的 JPR 变量的数据，或者获取一组偏移的 JPR 变量的数据。

原型

```
def get_jpr(handle, index = None, count = None) -> Union[Joint, List[Joint]]:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int, optional)`: JPR 变量序号(0~63)，不传时获取所有端口，传入时需与 `count` 组合使用
- `count (int, optional)`: 要获取的 JPR 数量(1~64)，仅当 `index` 存在时可指定，默认为 1

返回值

- `Union[Joint, List[Joint]]`: 单个 JPR 变量数值 或 JPR 变量数值列表

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 获取指定 Jpr 变量值
index=0
joint = x5.get_jpr(handle, index)
print(f"JPR0 变量数值: {joint}")
# 获取连续 JPR 变量值
index, count = 3, 4
joints = x5.get_jpr(handle, index, count)
for i in range(0, 4):
    print(f"JPR{3+i}变量数值: {joints[i]}")
# 获取所有 JPR 变量值
joints = x5.get_jpr(handle)
```

参数管理

坐标系

⚠ 警告

设置当前用户或者工具坐标系的序号功能，仅适用于手动模式，用于切换当前用户和工具序号并记录到点位。如运动时使用，则无法切换当前用户及工具序号。

set_ufno | 设置当前用户坐标系序号

设置当前正在使用的用户坐标系序号

原型

```
def set_ufno(handle, index) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: 用户坐标系序号(0~16)

执行条件

- 手动或自动命令模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# set_ufno
index = 11
x5.set_ufno(handle, index)
t.sleep(1)
```

get_ufno | 获取当前用户坐标系序号

获取当前正在使用的用户坐标系（UF）序号

原型

```
def get_ufno(handle) -> int:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `index (int)`: 用户坐标系序号(0~16)

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# get_ufno
index = x5.get_ufno(handle)
print(f"当前用户编号: {index}\n")
t.sleep(1)
```

set_tfno | 设置当前工具坐标系序号

设置当前正在使用的工具坐标系（TF）序号

原型

```
def set_tfno(handle, index) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: 工具坐标系序号(0~16)

执行条件

- 手动或自动命令模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# set_tfno
index = 11
x5.set_tfno(handle, index)
t.sleep(1)
```

get_tfno | 获取当前工具坐标系序号

获取当前正在使用的工具坐标系（TF）序号

原型

```
def get_tfno(handle) -> int:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `index (int)`: 工具坐标系序号(0~16)

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# get_tfno
index = x5.get_tfno(handle)
print(f"当前工具编号: {index}\n")
t.sleep(1)
```

set_uf | 设置用户坐标系数据

设置指定的用户坐标系数据（UF）

原型

```
def set_uf(handle, index, data) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: 用户坐标系序号(UF1~16)。不允许设置 UF0。
- `data (Pose)`: 位姿数据

执行条件

- 手动或自动命令模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# set_uf
index, data = 1, x5.Pose(10, 0, 0, 0, 0, 0, 0, 0, 0)
x5.set_uf(handle, index, data)
t.sleep(1)
```

get_uf | 获取用户坐标系数据

获取指定的用户坐标系数据（UF）

原型

```
def get_uf(handle, index) -> Pose:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: 用户坐标系序号(0~16)

返回值

- `pose (Pose)`: 用户坐标系数据

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

index= 1
# get_uf
data = x5.get_uf(handle, index)
print(f"UF 坐标: {data}\n")
t.sleep(1)
```

set_tf | 设置工具坐标系数据

设置指定的工具坐标系数据（TF）

原型

```
def set_tf(handle, index, data) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: 工具坐标系序号(TF1~16)。不允许设置 TF0。
- `data (Pose)`: 位姿数据

执行条件

- 手动或自动命令模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# set_tf
index, data = 1, x5.Pose(10, 0, 0, 0, 0, 0, 0, 0, 0)
x5.set_tf(handle, index, data)
t.sleep(1)
```

get_tf | 获取工具坐标系数据

获取指定的工具坐标系数据 (TF)

原型

```
def get_tf(handle, index) -> Pose:
```

参数

- `handle (int)`: 机器人句柄号
- `index (int)`: 工具坐标系序号(0~16)

返回值

- `pose (Pose)`: 工具坐标系数据

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

index = 1
# get_tf
data = x5.get_tf(handle, index)
print(f"TF 坐标: {data}\n")
t.sleep(1)
```

set_bf | 设置基坐标系数据

设置基坐标系的数据（BF）

原型

```
def set_bf(handle, pose) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `pose (Pose)`: 位姿数据

执行条件

- 手动或自动命令模式（建议手动模式下使用）

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# set_bf
data = x5.Pose(10, 0, 0, 0, 0, 0, 0, 0, 0)
x5.set_bf(handle, data)
t.sleep(1)
```

get_bf | 获取基坐标系数据

获取基坐标系的数据（BF）

原型

```
def get_bf(handle) -> Pose:
```

参数

- `handle (int)`: 机器人句柄号

返回值

- `pose (Pose)`: 基坐标系数据

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 获取 BF 坐标
data = x5.get_bf(handle)
print(f"BF 坐标: {data}\n")
```

calc_uf | 标定用户坐标系

通过 1 点、2 点、3 点法, 标定用户坐标系

函数原型:

```
def calc_uf(handle, poses, origin, tilt) -> Pose:
```

参数

- handle: int 型, 机器人句柄号。
- poses: list[Pose]型, 记录 Tcp 在世界(uf0)的坐标列表。标定点位个数 (1 点/2 点/3 点)
- origin: bool 型, 第一个点是否确认为坐标系原点 (False:不确认为原点, True:确认为原点)
- tilt: bool 型, 坐标系是否倾斜 (False:不倾斜, True:倾斜)

返回值

- Pose 型, 标定后的用户坐标系 (x,y,z,a,b,c)

执行条件

- 任意模式

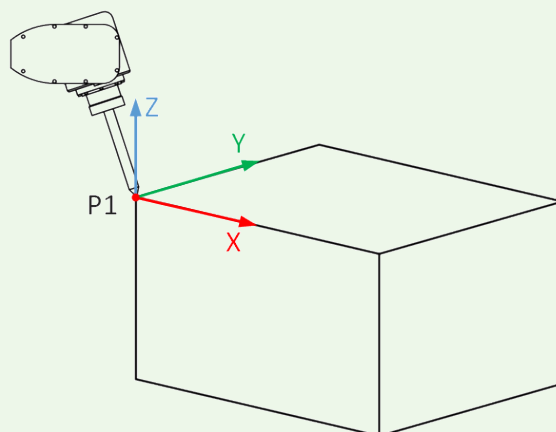
调用示例:

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: \n{handle}\n")

# 设定标定数据
origin = True
tilt = False
p1 = x5.Pose(200,200,0,0,0,0)
p2 = x5.Pose(250,200,0,0,0,0)
p3 = x5.Pose(250,250,0,0,0,0)
# 进行用户坐标系标定
result = x5.calc_uf(handle,[p1,p2,p3],origin,tilt)
print(f"\n 用户坐标: \n{result}\n")
# 输出:
# 用户坐标系
# x: 200.0; y: 200.0; z: 0.0; a: 0.0; b: -0.0; c: 0.0; e1: 0.0; e2: 0.0; e3: 0.0
```

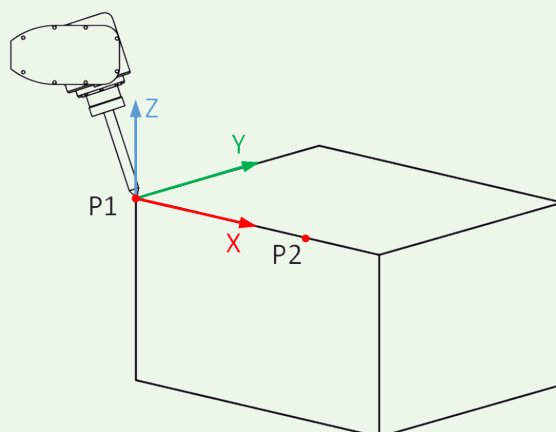
 备注

1 点标定：计算坐标系原点坐标相对于世界坐标系的偏移值，其方向与世界坐标系平行。



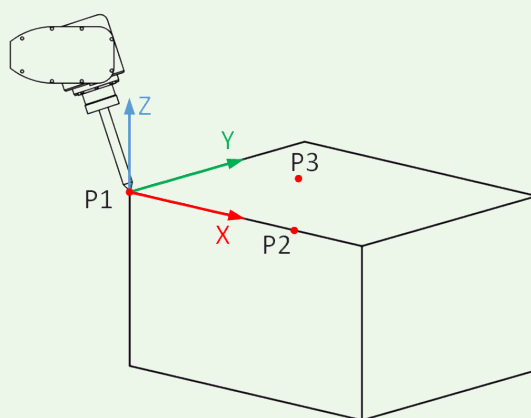
1. 将机器人 TCP 点手动移动到需要设定的坐标系的原点位置，记录当前工具在世界位置。
2. 标定出坐标系的原点坐标相对于世界坐标系的偏移值，其中 A、B、C 偏移值为 0；

2 点标定：计算出坐标系原点坐标相对于世界坐标系的偏移值，坐标系可确定 X 轴方向。

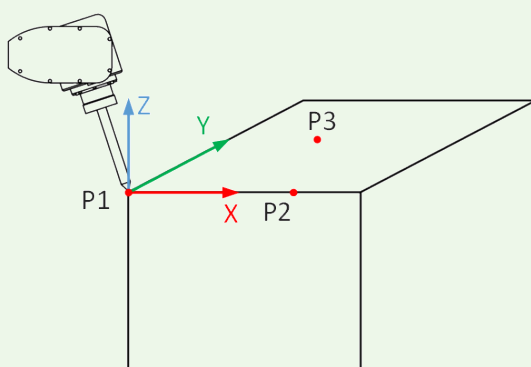


1. 将机器人 TCP 点手动移动到需要设定的坐标系的原点位置，记录当前工具在世界位置。
2. 记录 X 轴正方向，将机器人 TCP 点手动移动到下一点，确保移动后这一点与之前记录的原点之间的连线与要设定的坐标系的 X 轴正方向一致；
3. 标定出坐标系的原点坐标相对于世界坐标系的偏移值，其中 A 偏移值为 0；

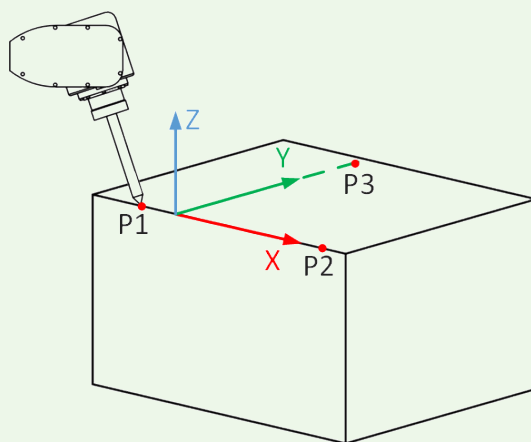
当 3 点标定时，需要确定是否原点，是否倾斜。当确认“原点”：



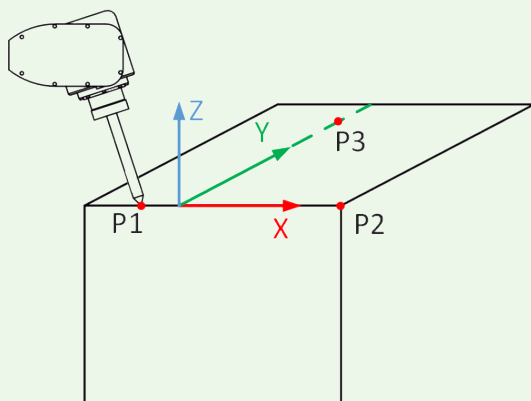
1. 记录 P1 点，首先将机器人的工具中心点（TCP）手动移动到需要设定坐标系的原点位置，记录当前工具在世界位置。
2. 记录 P2 点，继续将机器人的 TCP 点手动移动到下一个位置，确保之前记录的 P1 点与移动后这一点之间的连线与要设定的坐标系的 X 轴正方向一致。然后记录 P2 点的坐标数据，表示 X 轴正方向点已成功记录。
3. 记录 P3 点，将机器人的 TCP 点手动移动到下一个位置，确保移动到的点与之前记录 P2 点的连线处于要设定坐标系的 XY 平面内。然后记录 P3 点的坐标数据，表示 Y 轴正方向点已成功记录。
4. 标定出坐标系的原点坐标和相对于世界坐标系的旋转偏移值。用户坐标系的建立遵循右手法则，其中 Z 轴的正方向指向 X 轴向 Y 轴旋转的大拇指方向。（如需要计算倾斜角度，需要确认倾斜，即 tilt 为 true。系统会计算出 A、B 值）



3 点标定：计算出坐标系原点坐标相对于世界坐标系的偏移值，当不确认“原点”。



1. 记录 P1 点，首先将机器人的工具中心点（TCP）手动移动到需要设定坐标系的 X 轴上的任意位置，记录当前工具在世界位置。
2. 记录 P2 点，继续将机器人的 TCP 点手动移动到下一个位置，确保之前记录的 P1 点与移动后这一点之间的连线与要设定的坐标系的 X 轴正方向一致。然后记录 P2 点的坐标数据，表示 X 轴正方向点已成功记录。
3. 记录 P3 点，将机器人的 TCP 点手动移动到下一个位置，确保移动到的点与之前记录 P2 点的连线处于要设定坐标系的 XY 平面内。然后记录 P3 点的坐标数据，表示 Y 轴正方向点已成功记录。此时，原点位于 P3 点所在 X 轴垂线上的垂足位置。
4. 标定出坐标系的原点坐标和相对于世界坐标系的旋转偏移值。用户坐标系的建立遵循右手法则，其中 Z 轴的正方向指向 X 轴向 Y 轴旋转的大拇指方向。（如需要计算倾斜角度，需要确认倾斜，即 tilt 为 true。系统会计算出 A、B 值）



calc_tf | 标定工具坐标系

通过 3 点、4 点、5 点、6 点、8 点或 10 点法标定工具坐标系

函数原型：

```
def calc_tf(handle: int, poses: list[Pose]) -> tuple[Pose, list[float]]:
```

参数

- handle: int 型，机器人句柄号。
- poses: list[Pose]型，记录法兰在同一用户的坐标列表。点位个数：
 - 3-四自由度: 标定 XY 偏移量；
 - 4-四自由度/六自由度: 标定 XYZ 方向偏移量；
 - 5-四自由度: 标定 XY 偏移量和姿态 ABC；
 - 6-四自由度/六自由度: 标定 XYZ 方向偏移量和姿态 ABC；
 - 8-六自由度: 标定 XYZ 方向偏移量；
 - 10-六自由度: 标定 XYZ 方向偏移量和姿态 ABC；

返回值

- tuple[Pose, list[float]]:
 - Pose 型，标定后的工具坐标系 (x,y,z,a,b,c)；
 - bool 型，标定误差(err,0,0)；

执行条件

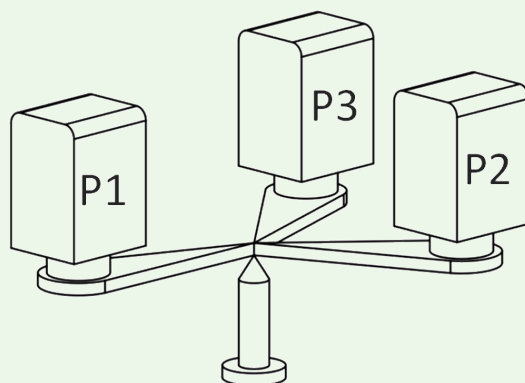
- 任意模式

调用示例：

```
import xapi.api as x5
# 创建连接
handle = x5.connect("192.168.1.6")
# 设定标定数据，记录 UF0TF0 下点位
p1 = x5.Pose(267,-98,-16,0,0,-100)
p2 = x5.Pose(178,-244,-16,0,0,14)
p3 = x5.Pose(336,-122,-16,0,0,-142)
# 进行工具坐标系标定
result = x5.calc_tf(handle,[p1,p2,p3])
print(f"\n 工具坐标: {result[0]}")
print(f"标定误差: {result[1][0]}\n")
# 输出:
# 工具坐标 x: 99.89379081360073; y: 20.608503289387695; z: 0; a: 0; b: 0; c: 0;
# 标定误差: 0.23890614912739397
```

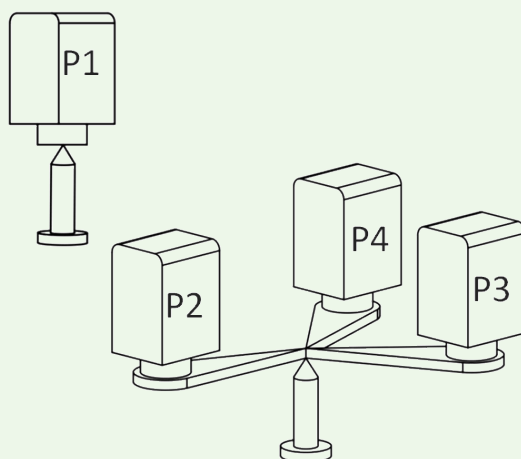
💡 备注

三点标定：计算出工具坐标系原点坐标相对于法兰坐标系的 XY 偏移值，方向与法兰平行。



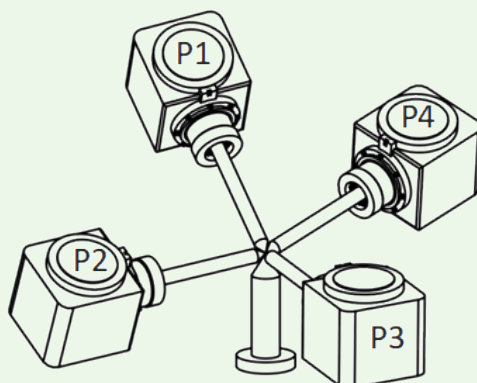
1. 将机器人 TCP 点分别手动移动到图示中标定杆的 3 个位置点，记录法兰在用户的坐标；记录 P2~P4 时，工具末端必须只有一个固定点，且其姿态变化要尽可能的大；
2. 标定出工具坐标系的原点坐标和旋转相对于法兰盘坐标系的偏移值，其 Z 偏移量为 0；

四点标定：计算出工具坐标系原点坐标相对于法兰坐标系的 XYZ 偏移值。机型（四轴）：



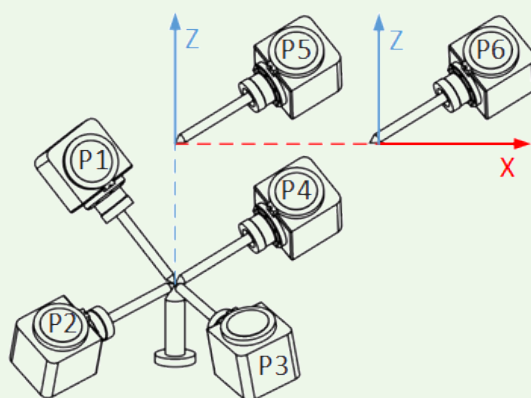
1. 将机器人 TCP 点分别手动移动到图示中标定杆的 4 个位置点，分别记录法兰在用户的坐标。其中 P1 点位置为机器人不带工具时，法兰面接触标定杆，而 P2~P4 点记录时，工具末端必须只有一个固定点，并且其姿态变化要尽可能的大；
2. 标定出该工具坐标系的原点坐标和旋转相对于法兰盘坐标系的偏移值；

四点标定：计算出工具坐标系原点坐标相对于法兰坐标系的 XYZ 偏移值。机型（六轴）：



1. 将机器人 TCP 点分别手动移动到图示中标定杆的 4 个位置点，分别记录法兰在用户的坐标。而 P1~P4 点记录时，工具末端必须只有一个固定点，且其姿态变化要尽可能大；
2. 标定出该工具坐标系的原点坐标和旋转相对于法兰盘坐标系的偏移值；

六点标定：计算出工具坐标系原点坐标相对于法兰坐标系的 XYZABC 偏移值。



1. 将机器人 TCP 点分别手动移动到图示中标定杆的 6 个位置点，分别记录法兰在用户的坐标。而 P1~P4 点记录时，工具末端必须只有一个固定点，且其姿态变化要尽可能大，而记录 P5 点时，工具末端为 Z 方向偏移，即 P4 与 P5 之间的连线为工具坐标系的 Z 轴正向，P6 点用来确定工具坐标系 X 轴正方向，即 P5 与 P6 之间的连线为工具坐标系的 X 轴正向；
2. 标定出该工具坐标系的原点坐标和旋转相对于法兰坐标系的偏移值；

calc_2p_tf | 目标点标定工具坐标系

根据已知目标点，标定工具坐标系

函数原型：

```
def calc_2p_tf(handle: int, target_pose: Pose, teach_pose: Pose) -> Pose:
```

参数

- handle: int 型，机器人句柄号。
- target_pose: Pose 型，目标点位。已知目标点位在用户的坐标 (x,y,z,a,b,c)
- teach_pose: Pose 型，示教点位。记录法兰(tf0)在同一用户的坐标 (x,y,z,a,b,c)

返回值

- Pose 型，标定后的工具坐标系 (x,y,z,a,b,c);

执行条件

- 任意模式

调用示例：

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: \n{handle}\n")

# 设定标定数据，记录 UF0 下已知目标点位 P1
p1 = x5.Pose(269.9,-200,-16,0,0,-132)
# 记录机器人带工具示教 P1 点时，UF0TF0 下点位 P2
p2 = x5.Pose(178,-244,-16,0,0,14)
# 进行工具坐标系标定
result = x5.calc_2p_tf(handle,p1,p2)
print(f"\n 工具坐标: {result}\n")
# 输出:
# 工具坐标: x: 99.81474; y: 20.46039; z: 0.0; a: 0.0; b: 0.0; c: -146.0
```

工艺应用：标定视觉工具 TF

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
```

```

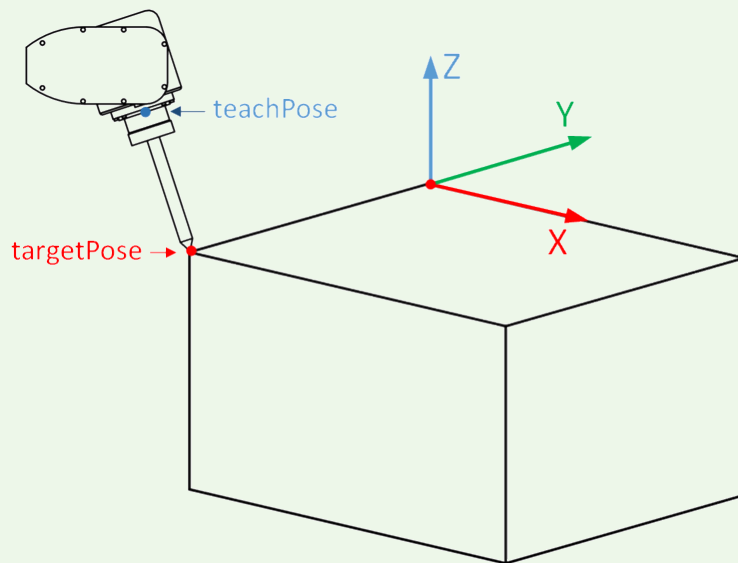
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: \n{handle}\n")

# 使用视觉工艺 0，计算视觉像素坐标(x=200, y=200, c=30)的静态视觉转换后的位姿数据。
pixel_data = x5.Pose(200,200,0,0,0,30,0,0,0)
pos = x5.vision_static_cnvt(handle, 0, pixel_data);
print(f"X={pos.x}, Y={pos.y}, C={pos.c}")
# 设定标定数据，记录用户下已知目标点位 P1
p1 = pos
# 记录机器人带工具示教 P1 点时，同一用户 TF0 下点位 P2
p2 = x5.Pose(178,-244,-16,0,0,14)
# 进行工具坐标系标定
result = x5.calc_2p_tf(handle,p1,p2)
print(f"\n 工具坐标: {result}\n")

```

💡 备注

根据已知目标点，标定工具坐标系详细操作：（如想了解更多坐标系标定，请联系厂商）



1. 将机器人 TCP 点手动移动到用户坐标系下，已知的目标点位置，记录目标点位置。
2. 记录当前机器人示教姿态下，机器人法兰在相同用户的坐标；
3. 计算出工具坐标系的原点坐标和旋转相对于法兰坐标系的偏移值；

cnvrt_j | 笛卡尔点位逆解求关节点位

笛卡尔点位逆解求关节点位

函数原型：

```
def cnvrt_j(handle, src_point, inverse_type, last_joint) -> Joint:
```

参数

- handle: int 型，机器人句柄号。
- src_point: Point 型，需逆解求关节坐标的笛卡尔点位
- inverse_type: int 型，模式选择参数(逆解类型)。0 代表 onlyconf, 1 代表 conf&turn, 2 代表 shortest
- last_joint: Joint 型，点位 src_point 的上一个关节点位

返回值

- Joint 型，逆解求出的目标关节点位

执行条件

- 任意模式

调用示例：

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: \n{handle}\n")
try:
    # 逆解，求笛卡尔点位 p1 的对应关节坐标
    p1 = x5.Point((200,200,0,0,0,0,0,0),0,0,(0,0,0,1))
    # 设定逆解类型及参考的关节点位
    type, joint = 0, x5.Joint(46,-1,39,180,0,0,0,0)
    # 逆解求 p1 的对应关节点位
    jp1 = x5.cnvrt_j(handle, p1, type, joint)
    print(jp1)
except x5.RobException as rex:
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
t.sleep(1)
```

cnvrt_p | 关节点位正解求笛卡尔点位

关节点位正解求笛卡尔点位

函数原型：

```
def cnvrt_p(handle, src_joint, uf, tf) -> Point:
```

参数

- handle: int 型，机器人句柄号。
- src_joint: Joint 型，需正解求笛卡尔点位的关节点位
- uf: int 型，用户坐标系编号 (0~16, 0 表示世界坐标系)
- tf: int 型，工具坐标系编号 (0~16, 0 表示法兰盘坐标系)

返回值

- Point 型，正解求出的目标笛卡尔点位

执行条件

- 任意模式

调用示例：

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: \n{handle}\n")
try:
    # 正解，求关节坐标 jp1，对应的笛卡尔点位
    jp1 = x5.Joint(0,90,0,300,0,0)
    # 设定正解点位所在用户及工具坐标系
    uf, tf = 0, 0
    # 正解求 jp1 的对应笛卡尔点位
    p1 = x5.cnvrt_p(handle, jp1, uf, tf)
    print(f"p1:{p1}")
except x5.RobException as rex:
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
t.sleep(1)
```

cnvrt_p_to_p | 笛卡尔点位坐标系转换

笛卡尔点位坐标系转换

函数原型：

```
def cnvrt_p_to_p(handle, source_point, target_uf, target_tf) -> Point:
```

参数

- handle: int 型，机器人句柄号。
- source_point: Point 型，源笛卡尔位置点位
- target_uf: int 型，转换后点位的用户坐标系编号 (0~16, 0 表示世界坐标系)
- target_tf: int 型，转换后点位的工具坐标系编号 (0~16, 0 表示法兰盘坐标系)

返回值

- Point 型，转换后的笛卡尔位置点位

执行条件

- 任意模式

调用示例：

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: \n{handle}\n")

try:
    # 设定源笛卡尔位置点位
    p1 = x5.Point((250,200,0,0,0,0,30,0,0),0,0,(0,0,0,1))
    # 设定转换后点位的坐标系编号
    uf, tf = 1, 1
    # 求转换后的笛卡尔位置点位
    p1 = x5.cnvrt_p_to_p(handle, p1, uf, tf)
    print(f"p1:{p1}")
except x5.RobException as rex:
    print(f"报错接口: {rex.api_name}")
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
```

工艺应用

视觉工艺

vision_static_cnvt | 计算静态视觉转换后的点位数据

计算并返回静态视觉转换后的位姿数据，用于将视觉像素坐标转换为位姿坐标。

函数原型：

```
def vision_static_cnvt(handle, index, pixel) -> Pose:
```

参数

- handle: int 型，机器人句柄号。
- index, int 型，视觉工艺序号(0~7)
- pixel, Pose 型，视觉像素坐标点的位姿数据

返回值

- Pose 型，经过动态视觉转换后的位姿数据

执行条件

- 任意模式，但要求对应序号的视觉工艺存在对应标定矩阵

调用示例：

```
import xapi.api as x5
import time as t

# ...
try:
    # 使用视觉工艺 0，计算视觉像素坐标(x=200, y=200, c=30)静态视觉转换的位姿数据。
    pixel_data = x5.Pose(200,200,0,0,0,30,0,0,0)
    pos = x5.vision_static_cnvt(handle, 0, pixel_data);
    print(f"X={pos.x}, Y={pos.y}, C={pos.c}")
    # 沿笛卡尔空间运动到视觉目标点
    pos = x5.Point((pos.x,pos.y,-10,0,0,pos.c,0,0,0),0,0,(0,0,0,1))
    x5.movel(handle, pos)
    x5.wait_move_done(handle)
except x5.RobException as rex:
    print(f"报错接口: {rex.api_name}")
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
```

vision_dynamic_cnvt | 计算动态视觉转换后的点位数据

计算并返回动态视觉转换后的位姿数据。

函数原型：

```
def vision_dynamic_cnvt(handle, index, pixel, trig_point) -> Pose:
```

参数

- handle: int 型，机器人句柄号。
- index, int 型，视觉工艺序号(0~7)
- pixel, Pose 型，视觉像素坐标点的位姿数据
- trig_point, Point 型，动态视觉拍照点的点位数据，用于辅助计算动态视觉转换

返回值

- Pose 型，经过动态视觉转换后的位姿数据

执行条件

- 任意模式，但要求对应序号的视觉工艺存在对应标定矩阵

调用示例：

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: \n{handle}\n")

try:
    # 使用视觉工艺 0，计算视觉像素坐标(x=200, y=200, c=30)动态视觉转换的位姿数据。
    pixel_data = x5.Pose(200,200,0,0,0,30,0,0,0)
    trig_point = x5.get_cpoint(handle);
    pos = x5.vision_dynamic_cnvt(handle, 0, pixel_data, trig_point);
    print(f"X={pos.x}, Y={pos.y}, C ={pos.c}")
    # 沿笛卡尔空间运动到视觉目标点
    pos = x5.Point((pos.x,pos.y,-10,0,0,pos.c,0,0,0),0,0,(0,0,0,1))
    x5.movel(handle, pos)
    x5.wait_move_done(handle)
except x5.RobException as rex:
    print(f"报错接口: {rex.api_name}")
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
```

高级接口

CMD 命令

execute_lua | 执行 lua 字符串命令(TODO)

执行机器人 lua 指令字符串命令

原型

```
def execute_lua(handle, cmd) -> None:
```

参数

- `handle (int)`: 机器人句柄号
- `cmd (str)`: lua 命令

执行条件

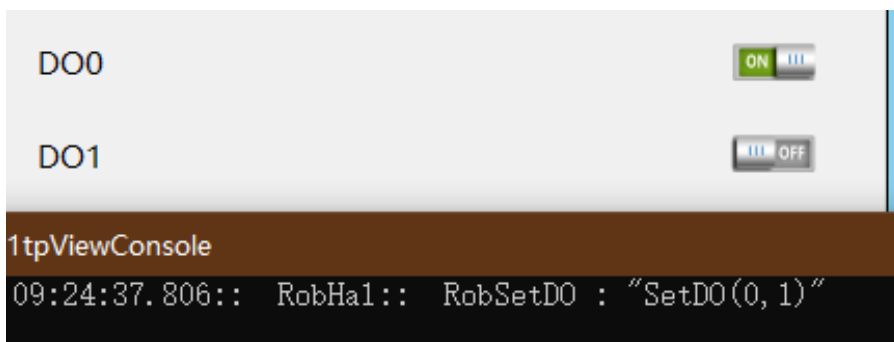
- 手动或者自动命令模式

示例

```
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 以字符串形式下发 CMD 命令
x5.execute_lua(handle, cmd="SetDO(0,1)")
t.sleep(1)
```

注意:

1. 操作 PC 示教器后(如打开 DO0)，控制台会返回执行的字符串命令 “SetDO(0,1)”，此命令即为 CmdStr 所传入的可执行字符串命令，如下图。



2. 仅支持如下 lua 指令：

运动指令：

MovJ, MovL, MovC, MovSJ, Jump, JumpL, MovSpl, MovArm, SetSpeed

流程指令：

Pause, Abort, DaeStart, DaePause, DaeAbort

坐标系指令：

SetUF, SetTF, SetBF, SetUFNo, SetTFNo, LockUF, UnlockUF, LockTF, UnlockTF

IO 指令：

SetDO, SetDOInvert, PulseDO, SetAO

变量指令：

SetModbusR, SetPlcData

其他指令：

OffsetCondition, ToolOffsetCondition, IgnoreOffset, IgnoreToolOffset,
SearchCondition, SetInverseType, Payload, AdaptTq, AdaptVel, ColDetect,
SetColLevel, LockArm, UnlockArm, Alarm, SetBlend

备注

lua 指令详细内容请参考《X5 机器人控制系统——lua 指令说明书》。

wait_cmd_send_done | 等待命令下发完成

等待上位机命令下发到机器人完成

原型

```
def wait_cmd_send_done(handle) -> None:
```

参数

- `handle (int)`: 机器人句柄号

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

x5.set_do(handle, 0, 1)
# 等待当前 CMD 命令发送完成
x5.wait_cmd_send_done(handle)
```

备注

该函数不保证机器人执行完成命令，仅保证命令已发送到机器人。

高级调试

enable_debug_output | 是否输出调试信息

是否输出高级调试信息

原型

```
def enable_debug_output(is_enable) -> None:
```

参数

- `enable (bool)`: True(输出), False(不输出)

执行条件

- 任意模式

示例

```
import xapi.api as x5
# 开启高级调试信息输出
x5.enable_debug_output(1)
# ... 中间代码
# 关闭高级调试信息输出
x5.enable_debug_output(0)
```

enable_read_message_thread | 是否开启字符串读取线程

是否开启字符串读取线程

原型

```
def enable_read_message_thread(handle, enable) -> None:
```

参数

- `handle (int)`: 机器人句柄号。
- `enable (bool)`: True-开启, False-关闭。

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")

# 开启字符串读取线程
x5.enable_read_message_thread(handle, 1)
# ... 中间代码
# 关闭字符串读取线程
x5.enable_read_message_thread(handle, 0)
```

get_lua_message | 获取 Lua 反馈字符串

获取 Lua 反馈字符串

原型

```
def get_lua_message(handle, read_buffer_size) -> tuple[str, int]:
```

参数

- `handle (int)`: 机器人句柄号。
- `read_buffer_size (int)`: 读取 buffer 大小。

返回值

- `value (tuple[str, int])`
 - `value[0] (str)`: 反馈字符串。
 - `value[1] (int)`: 剩余字符串数据长度。

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 开启字符串读取线程
x5.enable_read_message_thread(handle, 1)
while(True):
    results = x5.get_lua_message(handle, 128)
    # 如果字符串长度大于 0, 则打印字符串
    if len(results[0]) > 0:
        print(results[0])
        time.sleep(1)
# 关闭字符串读取线程
x5.enable_read_message_thread(handle, 0)
```

💡 备注

- 调用前需确保 `enable_read_message_thread()` 已开启。
- 若 `left_data_length` 大于 0, 则用户需继续执行该接口以获取剩余字符串。

get_last_error | 获取最后一次 API 错误

获取最后一次 API 错误

原型

```
def get_last_error() -> str:
```

返回值

- `value (str)`: 最后一次 API 错误。

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
try:
    handle = x5.connect(ip)
    print(f"\n 控制器句柄号: {handle}\n")
    x5.set_system_mode(handle, x5.SYSTEM_MODE_MANUAL)
    t.sleep(1)
    # 触发 run 接口模式错误
    x5.run(handle)
except x5.RobException as rex:
    print(f"报错接口: {rex.api_name}")
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
t.sleep(1)
try:
    # 获取最后一次 API 错误
    result = x5.get_last_error()
    # 如果字符串长度大于 0, 则打印字符串
    if len(result) > 0:
        print(len(result))
        print(result)
except x5.RobException as rex:
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
```

logger_get_recent_errors | 获取最近多条错误日志

获取最近多条错误日志

原型

```
def logger_get_recent_errors() -> list[str]:
```

返回值

- `list[str]` : 最近多条错误日志列表。

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# 出现多个接口报错，并被 try 捕获到
# ...
try:
    # 获取最近多条错误日志
    result = x5.logger_get_recent_errors()
    # 如果错误日志数量大于 0，则打印字符串
    if len(result) > 0:
        print(result)
except x5.RobException as rex:
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
```

备注

该接口时不可以在 `logger_shutdown` 后使用，否则将获取不到错误信息。

logger_shutdown | 释放日志缓存资源

释放日志缓存资源

原型

```
def logger_shutdown() -> None:
```

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
handle = x5.connect(ip)
print(f"\n 控制器句柄号: {handle}\n")
# ...
# 断开 TCP 连接
x5.disconnect(handle)
# 析构 API 日志
x5.logger_shutdown()
```

备注

API 内部会自动调用该接口以释放日志资源，若用户在主程序退出时遇到程序挂起问题，可手动调用该接口。

export_api_log_file | 导出 API 日志文件

导出 API 日志文件

原型

```
def export_api_log_file(zip_path) -> None:
```

参数

- `zip_path (str)`: 压缩包导出路径，如 “/path/to/xapi_logs.zip”。

执行条件

- 任意模式

示例

```
import xapi.api as x5
import time as t
# 控制器 IP 地址
ip = "192.168.1.6"
# 创建连接
try:
    handle = x5.connect(ip)
    print(f"\n 控制器句柄号: {handle}\n")
    # ...
    # 导出 API 日志文件
    zip_path = "xapi_logs.zip"
    x5.export_api_log_file(zip_path)
except x5.RobException as rex:
    print(f"报错接口: {rex.api_name}")
    print(f"错误代码: {rex.error_code}")
    print(f"错误内容: {rex.error_message}")
t.sleep(1)
```

附录

Ubuntu-Linux 实时环境配置

下载实时内核

1. 查看当前内核版本: `uname -a`
2. 下载实时内核和补丁 (选择和当前内核版本相近版本):
 - 内核: <https://mirror.tuna.tsinghua.edu.cn/kernel/v6.x/> (例如: linux-6.9.tar.gz)
 - 实时补丁: <https://mirror.tuna.tsinghua.edu.cn/kernel/projects/rt/6.9/> (例如: patch-6.9-rt5.patch.gz)
3. 解压内核: `tar -xzf linux-6.9.tar.gz`
4. 进入解压后的目录: `cd linux-6.9`
5. 打补丁: `gzip -cd ../patch-6.9-rt5.patch.gz | patch -p1 --verbose`

下载依赖

1. 安装依赖: `sudo apt-get install build-essential libncurses-dev libssl-dev libelf-dev bison flex`

编译安装内核

1. 配置内核: `make menuconfig` 进入 General setup -> Preemption model, 选择 Full Preemption Kernel (RT)。
2. 编辑 `.config` 文件, 修改以下内容:

```
CONFIG_SYSTEM_TRUSTED_KEY=""  
CONFIG_SYSTEM_REVOCATION_KEYS=""
```

2. 编译内核: `make -j$(nproc)`
3. 安装内核: `sudo make modules_install install`
4. 配置 grub:
 1. 编辑 `/etc/default/grub` 文件, 修改以下内容:

```
# GRUB_TIMEOUT_STYLE=hidden  
GRUB_TIMEOUT=10
```

2. 生成 grub 配置文件: `sudo update-grub`
3. 重启系统: `sudo reboot`

显卡在实时内核下安装驱动解决方法：

在非实时内核下

1. 根据显卡型号，下载对应的驱动
2. 安装 dkms 管理工具 `sudo apt-get install dkms`
3. 禁用 nouveau `sudo gedit /etc/modprobe.d/blacklist.conf`, 在文件中最后一行加入

```
blacklist nouveau
options nouveau modeset=0
```

保存退出，在终端输入 `sudo update-initramfs -u`，然后重启

4. 卸载之前的驱动 `sudo apt-get remove --purge nvidia*`
5. 安装新的驱动
6. 重启

在实时内核下利用 sh 文件安装驱动

1. 创建一个 sh 文件，例如 `install_nvidia.sh`，内容如下：

```
#!/bin/bash
BUILD_BASE=`pwd`
NV_FILE="NVIDIA-Linux-x86_64-xxx.run" # 这里改成自己下载的.run 文件名（之前安装过的）
#NV_URL="https://us.download.nvidia.cn/XFree86/Linux-x86_64/430.50/${NV_FILE}" # 之前已经下好了显卡驱动.run 文件，就不用从网上下载了，直接注释掉，而且如果要下载的话，则慢
NEED_TO_COMPILE_NV_KO=1

function clean_env() {
    [ -d ./${NV_DIR} ] && rm -rf ./${NV_DIR}
}

function check_env() {
    # check if in rt kernel
    uname -r | grep rt 1>/dev/null 2>&1
    if [ $? -ne 0 ]
    then
        echo "Not in rt kernel, Please install apollo kernel and reboot machine first."
        exit 2
    fi
}
```

```

# check if nv ko already in kernel
if [ ! -f /lib/modules/`uname -r`/kernel/drivers/video/nvidia.ko ]
then
    export NEED_TO_COMPILE_NV_KO=1
fi
}

function prepare_nv() {

    ## download nv install file from nvidia home page
    #if [ ! -f ./${NV_FILE} ]
    #then
    #    echo "Downloading ${NV_FILE} from nvidia website..."
    #    wget ${NV_URL} -O ${NV_FILE}
    #    if [ $? -ne 0 ]
    #    then
    #        echo "Downloading ${NV_FILE} failed, please check your network
connection!"
    #        rm -rf ./${NV_FILE}
    #        exit 1
    #    fi
    #fi

    ##### 上面是下载驱动的代码，我们已经提前下载好了，就不需要这段代码了，直接
注释掉 #####

    # +x
    chmod +x ./${NV_FILE}
    echo "Extracting nvidia install run file..."
    ./${NV_FILE} -x 1>/dev/null 2>&1
    NV_DIR="`echo ${NV_FILE} | awk -F '.run' '{print $1}'`"
    NV_VERSION="`echo ${NV_FILE} | awk -F '-' '{print $4}' | awk -F '.run' '{print
$1}'`"

    export NV_DIR
    export NV_VERSION
    export NVIDIA_SOURCE="${NV_DIR}/kernel"
}

function install_lib() {

    NV_LIB_OUTPUT_PATH="/usr/lib/x86_64-linux-gnu/"
    NV_BIN_OUTPUT_PATH="/usr/bin/"

```

```

[ -f ./${NV_DIR}/libnvidia-ml.so.${NV_VERSION} ] && /bin/cp -f ./${NV_DIR}/
libnvidia-ml.so.${NV_VERSION} ${NV_LIB_OUTPUT_PATH}
[ -f ./${NV_DIR}/libnvidia-fatbinaryloader.so.${NV_VERSION} ] && /bin/cp -f ./
${NV_DIR}/libnvidia-fatbinaryloader.so.${NV_VERSION} ${NV_LIB_OUTPUT_PATH}
[ -f ./${NV_DIR}/libnvidia-ptxjitcompiler.so.${NV_VERSION} ] && /bin/cp -f ./
${NV_DIR}/libnvidia-ptxjitcompiler.so.${NV_VERSION} ${NV_LIB_OUTPUT_PATH}
[ -f ./${NV_DIR}/libcuda.so.${NV_VERSION} ] && /bin/cp -f ./${NV_DIR}/
libcuda.so.${NV_VERSION} ${NV_LIB_OUTPUT_PATH}
[ -f ./${NV_DIR}/nvidia-modprobe ] && /bin/cp -f ./${NV_DIR}/nvidia-modprobe
${NV_BIN_OUTPUT_PATH}
[ -f ./${NV_DIR}/nvidia-smi ] && /bin/cp -f ./${NV_DIR}/nvidia-smi
${NV_BIN_OUTPUT_PATH}

chmod +x /usr/bin/nvidia*
chmod +s /usr/bin/nvidia-modprobe

# link for nvidia
/bin/rm -rf /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1 /usr/lib/x86_64-linux-
gnu/libnvidia-ml.so
/bin/ln -s /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.${NV_VERSION} /usr/lib/
x86_64-linux-gnu/libnvidia-ml.so.1
/bin/ln -s /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1 /usr/lib/x86_64-linux-
gnu/libnvidia-ml.so

/bin/rm -rf /usr/lib/x86_64-linux-gnu/libcuda.so /usr/lib/x86_64-linux-gnu/
libcuda.so.1
/bin/ln -s /usr/lib/x86_64-linux-gnu/libcuda.so.${NV_VERSION} /usr/lib/x86_64-
linux-gnu/libcuda.so.1
/bin/ln -s /usr/lib/x86_64-linux-gnu/libcuda.so.1 /usr/lib/x86_64-linux-gnu/
libcuda.so
# take effect
/sbin/ldconfig 1>/dev/null 2>&1
}

function build_nv() {

if [ ${NEED_TO_COMPILE_NV_KO} == 0 ]
then
    return
fi

NVIDIA_MOD_REL_PATH='kernel/drivers/video'

```

```

NVIDIA_OUTPUT_PATH="/lib/modules/`uname -r`/${NVIDIA_MOD_REL_PATH}"
CPUNUM=`cat /proc/cpuinfo | grep processor | wc | awk -F " " '{print $1}'`

export IGNORE_PREEMPT_RT_PRESENCE=true
cd ${NVIDIA_SOURCE} && make -j ${CPUNUM} module
cd ${BUILD_BASE}

unset IGNORE_PREEMPT_RT_PRESENCE

mkdir -p ${NVIDIA_OUTPUT_PATH}

[ -f ${NVIDIA_SOURCE}/nvidia.ko ] && cp ${NVIDIA_SOURCE}/nvidia.ko
${NVIDIA_OUTPUT_PATH}
[ -f ${NVIDIA_SOURCE}/nvidia-modeset.ko ] && cp ${NVIDIA_SOURCE}/nvidia-
modeset.ko ${NVIDIA_OUTPUT_PATH}
[ -f ${NVIDIA_SOURCE}/nvidia-drm.ko ] && cp ${NVIDIA_SOURCE}/nvidia-drm.ko
${NVIDIA_OUTPUT_PATH}
[ -f ${NVIDIA_SOURCE}/nvidia-uvm.ko ] && cp ${NVIDIA_SOURCE}/nvidia-uvm.ko
${NVIDIA_OUTPUT_PATH}

depmod -a
}

# check environment
check_env

# prepare for nvidia
prepare_nv

# build nvidia.ko
build_nv

# install user lib
install_lib

# clean environment
clean_env

echo "Done to install nvidia kernel driver and user libraries."

```

2、赋予执行权限 `chmod +x install_nvidia.sh`

3、执行 `./install_nvidia.sh`

4、重启

日志功能

备注

API 日志功能默认关闭，如想使用日志功能，请修改 xapi.ini 配置文件！

运行日志

运行 API 接口连接到机器人之后，API 会在调用各类操作接口时打印日志到文件，包含调用参数和返回结果等信息。可在遇到问题时将日志提供给公司的技术支持人员，以便分析排查。目前暂不支持用户自定义配置日志的输出位置，默认的设置如下：

- 日志文件路径：<库文件夹>/Lib/site-packages/xapi/libs/64bit/xapi/xlog/xlog.log
- 配置文件路径：<库文件夹>/Lib/site-packages/xapi/libs/64bit/xapi/xapi.ini
- 最多保留 10 个日志文件

目前支持用户自定义配置输出日志的内容，需要手动修改 xapi.ini 文件，详细内容如下：

```
[xlog]
; XAPI 日志以配置文件驱动方式的失效日期(纯数字, 格式:YYYYMMDD):20250101 (2025年1月1日失效, 仅在日志初始化
expiration_date = 20260101
; 控制台日志记录级别(纯数字):0-off, 1-trace, 2-debug, 3-info, 4-warning, 5-err, 6-critical = 默认:0
console_log_level = 2
; 文件日志记录级别(纯数字):0-off, 1-trace, 2-debug, 3-info, 4-warning, 5-err, 6-critical = 默认:0
file_log_level = 2
; 文件日志最大大小(纯数字):最小1024[字节] = 默认:1048576(1M)
file_log_max_size = 1048576
; 文件日志最多轮转数量(纯数字):最少1 = 默认:10
file_log_max_count = 10
; 重复日志相邻比对条数(纯数字):0~5 = 默认:0
duplicate_log_comparison_count = 0
; 错误日志回溯栈大小(纯数字):0~20 = 默认:20
error_log_backtrace_size = 20
```

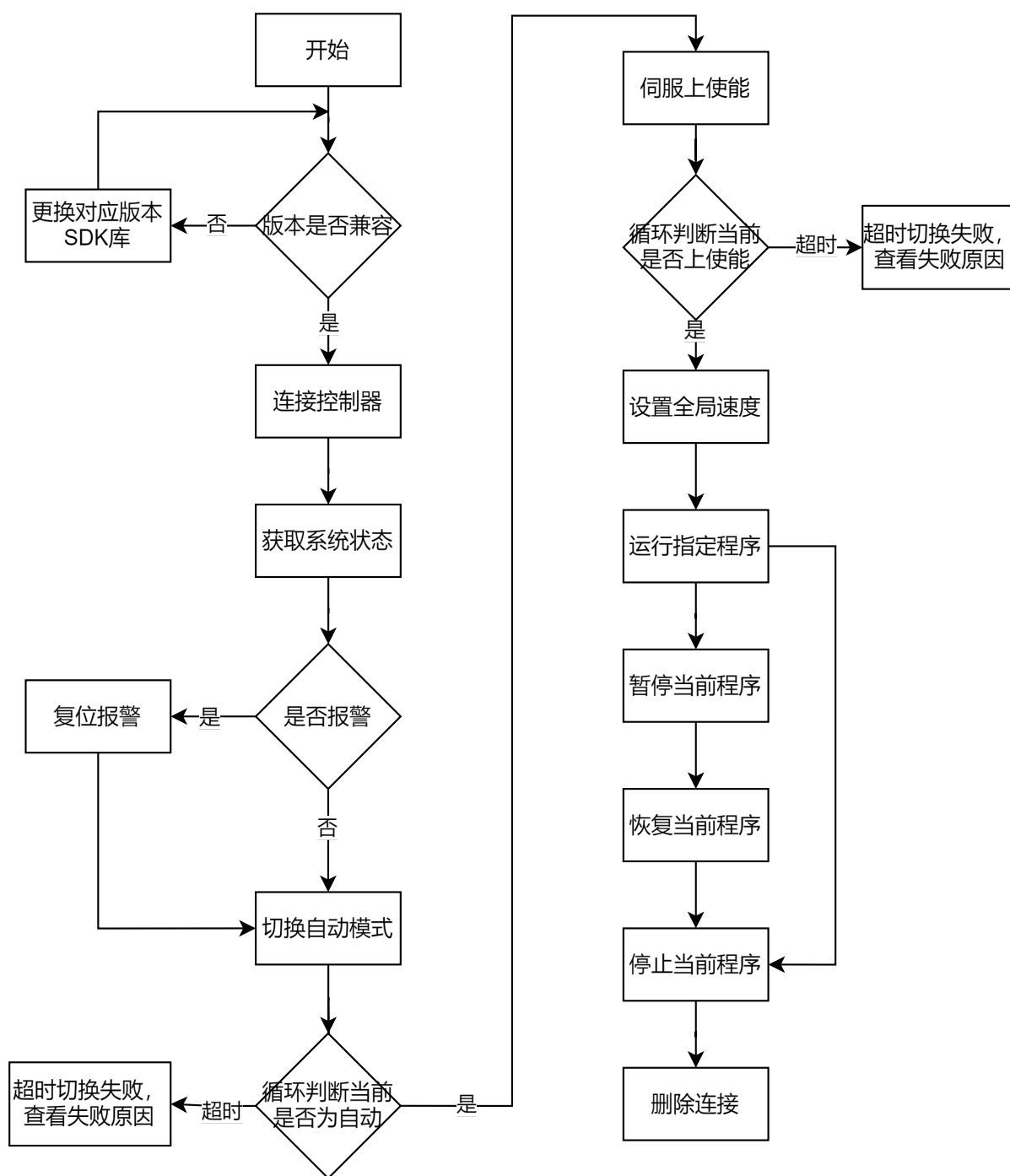
- 日志配置文件失效日期：当前日期小于失效日期，则日志配置文件生效，开启日志功能。
- 控制台日志记录级别：

设置控制台输出调试信息的内容级别，而接口 enable_debug_output 控制是否输出到控制台。默认 0 关闭，需要调试使用时，建议设置为 2。
- 文件日志记录级别：设置文件中记录的日志内容级别，需要调试使用时，建议设置为 2。
- 文件日志最大大小：设置日志文件的大小，建议使用默认值。
- 文件日志最多轮转数量：设置记录日志文件的数量，建议使用默认值。
- 重复日志相邻比对条数：设置重复条数过滤的数量，建议使用默认值。
- 错误日志回溯栈大小：设置记录最近的错误数量，建议使用默认值。

例行程序

上位机控制机器人自动运行指定程序

在上位机控制机器人自动运行程序前，请确保该程序已编译完成且程序编写正确可以正常运行。
流程如下：



代码实现：

```

import time
import xapi.api as x5

class CheckType:
    RunState = 1
    Mode = 2
    InPos = 3
    Alarm = 4
    Remote = 5
    Enable = 6
    CmdQueFull = 7
    LinkStat = 8
    Speed = 9

# 常量定义
IP = "168.168.110.30"
# test1.xml 或者 test1.lua 程序
PROGRAM_NAME = "test1"
GLOBAL_SCALE = 5
MAX_WAIT_INTERVAL_MS = 2000
MAX_WAIT_CYCLE_TIMES = 30
WAIT_INTERVAL_MS = 100

# 设置暂停和恢复标志
is_pause_pro_flag = True

# 创建新的连接
try:
    handle = x5.connect(IP)
    print(f"连接至机器人[{IP}]")
except x5.RobException as ex:
    print(f"连接机器人[{IP}]失败！")
    print(f"错误代码：{ex.error_code}")
    print(f"错误信息：{ex.error_message}")
    if ex.error_code != -200: # -200: 重复连接至同一控制器
        # 用户定义的处理方式
        pass

def init_sys_err(handle):
    """初始化系统错误"""
    try:
        state = x5.get_system_state(handle)

```

```

    if state.alarm:
        x5.reset(handle)
        x5.wait_cmd_send_done(handle)
    wait_for_system_state(handle, CheckType.Mode, 0)
    print("清除报警状态成功")
except x5.RobException as ex:
    print("清除报警状态失败！")
    print(f"错误代码：{ex.error_code}")
    print(f"错误信息：{ex.error_message}")

def set_sys_remote(handle, type):
    """设置系统远程模式"""
    try:
        state = x5.get_system_state(handle)
        if state.remote != bool(type):
            x5.set_remote(not state.remote)
        x5.wait_cmd_send_done(handle)
        wait_for_system_state(handle, CheckType.Mode, type)
        print(f"设置{'远程' if type == 1 else '本地'}模式成功")
    except x5.RobException as ex:
        print(f"设置{'远程' if type == 1 else '本地'}模式失败！")
        print(f"错误代码：{ex.error_code}")
        print(f"错误信息：{ex.error_message}")

def set_sys_mode(handle, mode):
    """设置系统模式"""
    try:
        state = x5.get_system_state(handle)
        if state.mode != mode:
            if mode == 2:
                x5.set_system_mode(handle, x5.SYSTEM_MODE_AUTO)
            elif mode == 3:
                x5.set_system_mode(handle, x5.SYSTEM_MODE_MANUAL)
            elif mode == 5:
                x5.set_system_mode(handle, x5.SYSTEM_MODE_DEBUG)
            else:
                x5.set_system_mode(handle, x5.SYSTEM_MODE_AUTOCMD)
        x5.wait_cmd_send_done(handle)
        wait_for_system_state(handle, CheckType.Mode, mode)
        print(f"切换{state.mode}模式成功")
    except x5.RobException as ex:
        print(f"切换{state.mode}模式失败！")
        print(f"错误代码：{ex.error_code}")

```

```

        print(f"错误信息: {ex.error_message}")

def set_sys_servo(handle, servo):
    """设置系统伺服"""
    try:
        state = x5.get_system_state(handle)
        if state.enable != bool(servo):
            x5.enable_servo(handle, not state.enable)
        x5.wait_cmd_send_done(handle)
        wait_for_system_state(handle, CheckType.Enable, servo)
        print(f"设置{'上使能' if servo == 1 else '下使能'}模式成功")
    except x5.RobException as ex:
        print(f"设置{'上使能' if servo == 1 else '下使能'}模式失败!")
        print(f"错误代码: {ex.error_code}")
        print(f"错误信息: {ex.error_message}")

def set_sys_speed(handle, speed):
    """设置系统速度"""
    try:
        x5.set_speed(handle, speed)
        x5.wait_cmd_send_done(handle)
        wait_for_system_state(handle, CheckType.Speed, speed)
        print(f"设置速度({speed}%)成功")
    except x5.RobException as ex:
        print(f"设置速度({speed}%)失败!")
        print(f"错误代码: {ex.error_code}")
        print(f"错误信息: {ex.error_message}")

def wait_for_system_state(handle, type, expectant):
    """等待系统状态变化"""
    times = 1
    is_error = True
    try:
        while True:
            state = x5.get_system_state(handle)
            check_result = True
            if type == CheckType.RunState:
                check_result = state.run == expectant
            elif type == CheckType.Mode:
                check_result = state.mode == expectant
            elif type == CheckType.InPos:
                check_result = int(state.in_pos) == expectant
            elif type == CheckType.Alarm:

```

```

        check_result = int(state.alarm) == expectant
    elif type == CheckType.Remote:
        check_result = int(state.remote) == expectant
    elif type == CheckType.Enable:
        check_result = int(state.enable) == expectant
    elif type == CheckType.CmdQueFull:
        check_result = state.cmd_que == expectant
    elif type == CheckType.LinkStat:
        check_result = int(state.link) == expectant
    elif type == CheckType.Speed:
        speed = x5.get_speed(handle)
        check_result = speed == expectant
    if check_result:
        break
    if times == MAX_WAIT_CYCLE_TIMES:
        is_error = False
        break
    times += 1
    time.sleep(WAIT_INTERVAL_MS / 1000.0)
except x5.RobException as ex:
    print(f"错误代码: {ex.error_code}")
    print(f"错误信息: {ex.error_message}")
    is_error = False
return is_error

def release_resources(handle):
    """释放资源"""
    try:
        x5.enable_servo(handle, False)
        wait_for_system_state(handle, CheckType.Enable, 0)
        print("设置下使能成功")
        x5.disconnect(handle)
        print(f"断开至机器人[{IP}]")
    except x5.RobException as ex:
        print("设置下使能失败!")
        print(f"错误代码: {ex.error_code}")
        print(f"错误信息: {ex.error_message}")

# 初始化系统错误
init_sys_err(handle)
# 设置系统为本地模式
set_sys_remote(handle, 0)
# 设置系统为自动模式

```

```

set_sys_mode(handle, 2) # Auto mode
# 设置系统伺服为上使能模式
set_sys_servo(handle, 1) # Enable servo
# 设置系统速度
set_sys_speed(handle, GLOBAL_SCALE)

# 运行指定程序
try:
    x5.run(handle, PROGRAM_NAME)
    x5.wait_cmd_send_done(handle)
    print(f"运行程序[{PROGRAM_NAME}]命令下发成功")
except x5.RobException as ex:
    print(f"运行程序[{PROGRAM_NAME}]命令执行失败！")
    print(f"错误代码：{ex.error_code}")
    print(f"错误信息：{ex.error_message}")

time.sleep(MAX_WAIT_INTERVAL_MS / 1000.0)

# 如果需要暂停和恢复程序
if is_pause_pro_flag:
    try:
        x5.pause(handle)
        x5.wait_cmd_send_done(handle)
        print("暂停程序命令下发成功")
    except x5.RobException as ex:
        print("暂停程序命令执行失败！")
        print(f"错误代码：{ex.error_code}")
        print(f"错误信息：{ex.error_message}")

    time.sleep(MAX_WAIT_INTERVAL_MS / 1000.0)

    try:
        x5.resume(handle)
        x5.wait_cmd_send_done(handle)
        print("恢复程序命令下发成功")
    except x5.RobException as ex:
        print("恢复程序命令执行失败！")
        print(f"错误代码：{ex.error_code}")
        print(f"错误信息：{ex.error_message}")

    time.sleep(MAX_WAIT_INTERVAL_MS / 1000.0)

# 停止运行程序

```

```
try:
    x5.stop(handle)
    x5.wait_cmd_send_done(handle)
    print("停止程序命令下发成功")
except x5.RobException as ex:
    print("停止程序命令执行失败！")
    print(f"错误代码：{ex.error_code}")
    print(f"错误信息：{ex.error_message}")

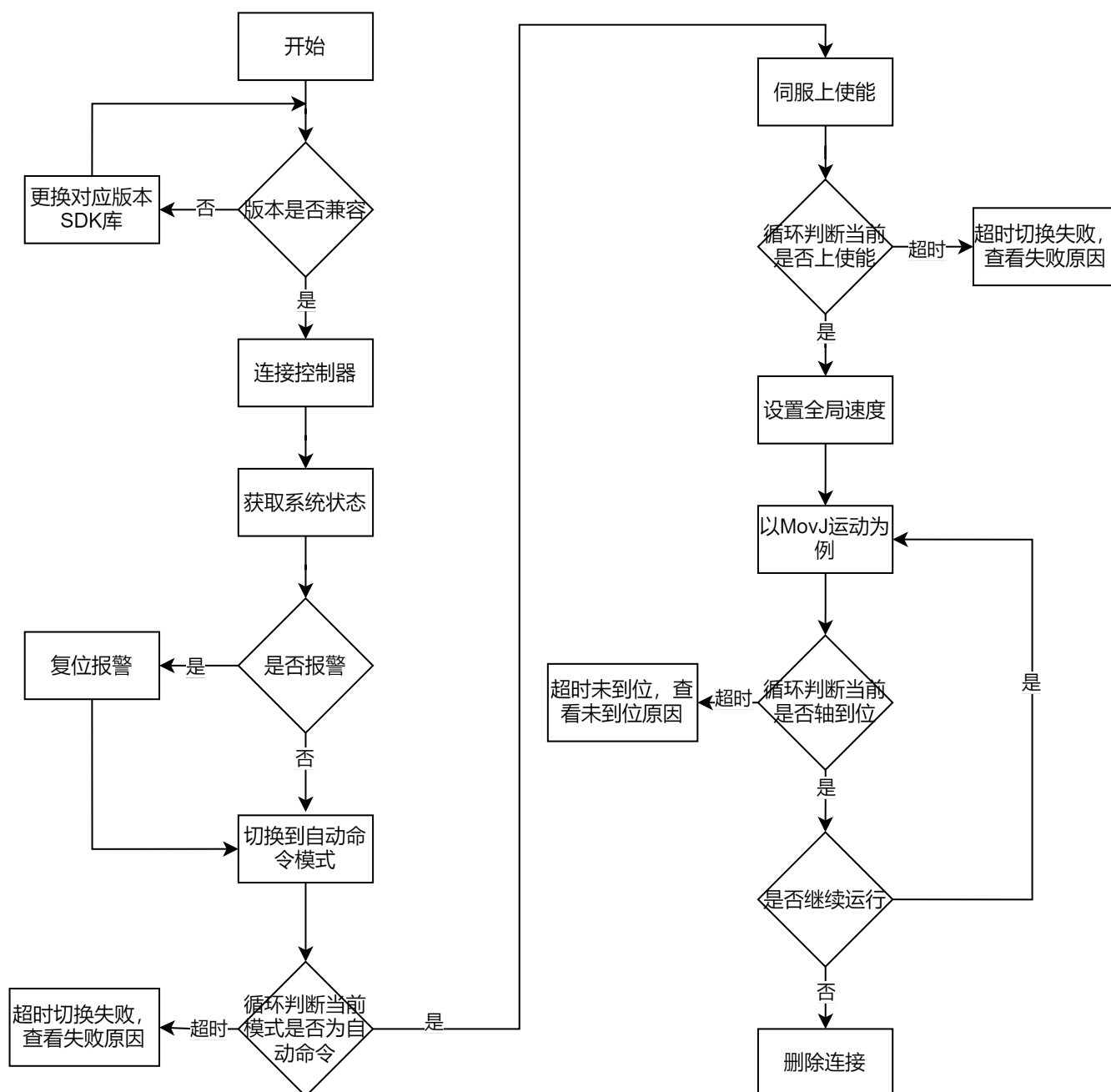
# 等待系统运行状态停止
time.sleep(2)

# 释放资源
release_resources(handle)
print("***结束***")
```

上位机控制机器人运行点位

在上位机控制机器人运行点位前，请确保点位格式正确且在机器人的行程之内。

流程如下：



代码实现：

```

import time
import xapi.api as x5

class CheckType:
    RunState = 1 # 系统运行状态
    Mode = 2 # 系统模式
    InPos = 3 # 到位状态
    Alarm = 4 # 报警状态
    Remote = 5 # 远程模式状态
    Enable = 6 # 使能状态
    CmdQueFull = 7 # 命令队列满状态
    LinkStat = 8 # 连接状态
    Speed = 9 # 速度

# 常量
IP = "168.168.110.30" # 机器人控制器 IP 地址
GLOBAL_SCALE = 5 # 全局速度 (1~100%)
MAX_WAIT_INTERVAL_MS = 2000 # 最大等待时间 (毫秒)
MAX_WAIT_CYCLE_TIMES = 30 # 最大等待周期数
WAIT_INTERVAL_MS = 100 # 间隔时间 (毫秒)

# 定义关节点位置
JP1 = x5.Joint(j1=40, j2=-90, j3=0, j4=0, j5=0, j6=0, e1=0, e2=0, e3=0)
JP2 = x5.Joint(j1=100, j2=-90, j3=0, j4=0, j5=0, j6=0, e1=0, e2=0, e3=0)

# 启用调试输出
x5.enable_debug_output(False)

# 创建一个新的连接
try:
    handle = x5.connect(IP)
    print(f"连接至机器人[{IP}]")
except x5.RobException as ex:
    print(f"连接机器人[{IP}]失败！")
    print(f"错误代码：{ex.error_code}")
    print(f"错误信息：{ex.error_message}")
    if ex.error_code != -200: # -200: 重复连接至同一控制器
        # 用户自定义处理
        pass

# 初始化系统报警
def init_sys_err(handle):

```

```

try:
    state = x5.get_system_state(handle)
    if state.alarm:
        x5.reset(handle)
        x5.wait_cmd_send_done(handle)
    wait_for_system_state(handle, CheckType.Mode, 0)
    print("清除报警状态成功")
except x5.RobException as ex:
    print("清除报警状态失败！")
    print(f"错误代码：{ex.error_code}")
    print(f"错误信息：{ex.error_message}")

# 设置系统远程/本地模式
def set_sys_remote(handle, type):
    try:
        state = x5.get_system_state(handle)
        if state.remote != bool(type):
            x5.set_remote(not state.remote)
        x5.wait_cmd_send_done(handle)
        wait_for_system_state(handle, CheckType.Mode, type)
        print(f"设置{'远程' if type == 1 else '本地'}模式成功")
    except x5.RobException as ex:
        print(f"设置{'远程' if type == 1 else '本地'}模式失败！")
        print(f"错误代码：{ex.error_code}")
        print(f"错误信息：{ex.error_message}")

# 设置系统模式
def set_sys_mode(handle, mode):
    try:
        state = x5.get_system_state(handle)
        if state.mode != mode:
            if mode == 2:
                x5.set_system_mode(handle, x5.SYSTEM_MODE_AUTO)
            elif mode == 3:
                x5.set_system_mode(handle, x5.SYSTEM_MODE_MANUAL)
            elif mode == 5:
                x5.set_system_mode(handle, x5.SYSTEM_MODE_DEBUG)
            else:
                x5.set_system_mode(handle, x5.SYSTEM_MODE_AUTOCMD)
        x5.wait_cmd_send_done(handle)
        wait_for_system_state(handle, CheckType.Mode, mode)
        print(f"切换{state.mode}模式成功")
    except x5.RobException as ex:

```

```

        print(f"切换{state.mode}模式失败！")
        print(f"错误代码：{ex.error_code}")
        print(f"错误信息：{ex.error_message}")

# 设置系统使能状态
def set_sys_servo(handle, servo):
    try:
        state = x5.get_system_state(handle)
        if state.enable != bool(servo):
            x5.enable_servo(handle, not state.enable)
        x5.wait_cmd_send_done(handle)
        wait_for_system_state(handle, CheckType.Enable, servo)
        print(f"设置{'上使能' if servo == 1 else '下使能'}模式成功")
    except x5.RobException as ex:
        print(f"设置{'上使能' if servo == 1 else '下使能'}模式失败！")
        print(f"错误代码：{ex.error_code}")
        print(f"错误信息：{ex.error_message}")

# 设置系统速度
def set_sys_speed(handle, speed):
    try:
        x5.set_speed(handle, speed)
        x5.wait_cmd_send_done(handle)
        wait_for_system_state(handle, CheckType.Speed, speed)
        print(f"设置速度({speed}%)成功")
    except x5.RobException as ex:
        print(f"设置速度({speed}%)失败！")
        print(f"错误代码：{ex.error_code}")
        print(f"错误信息：{ex.error_message}")

# 等待系统状态匹配预期值
def wait_for_system_state(handle, type, expectant):
    times = 1
    is_error = True
    try:
        while True:
            state = x5.get_system_state(handle)
            check_result = True
            if type == CheckType.RunState:
                check_result = state.run == expectant
            elif type == CheckType.Mode:
                check_result = state.mode == expectant
            elif type == CheckType.InPos:

```

```

        check_result = int(state.in_pos) == expectant
    elif type == CheckType.Alarm:
        check_result = int(state.alarm) == expectant
    elif type == CheckType.Remote:
        check_result = int(state.remote) == expectant
    elif type == CheckType.Enable:
        check_result = int(state.enable) == expectant
    elif type == CheckType.CmdQueFull:
        check_result = state.cmd_que == expectant
    elif type == CheckType.LinkStat:
        check_result = int(state.link) == expectant
    elif type == CheckType.Speed:
        speed = x5.get_speed(handle)
        check_result = speed == expectant
    if check_result:
        break
    if times == MAX_WAIT_CYCLE_TIMES:
        is_error = False
        break
    times += 1
    time.sleep(WAIT_INTERVAL_MS / 1000.0)
except x5.RobException as ex:
    print(f"错误代码: {ex.error_code}")
    print(f"错误信息: {ex.error_message}")
    is_error = False
return is_error

# 释放资源
def release_resources(handle):
    try:
        x5.enable_servo(handle, False)
        wait_for_system_state(handle, CheckType.Enable, 0)
        print("设置下使能成功")
        x5.disconnect(handle)
        print(f"断开至机器人[{IP}]")
    except x5.RobException as ex:
        print("设置下使能失败!")
        print(f"错误代码: {ex.error_code}")
        print(f"错误信息: {ex.error_message}")

# Main Program
init_sys_err(handle)
set_sys_remote(handle, 0) # 设置为本地模式

```

```
set_sys_mode(handle, 100) # 设置为自动命令模式
set_sys_servo(handle, 1) # 设置为上使能状态
set_sys_speed(handle, GLOBAL_SCALE) # 设置全局速度

# 执行关节运动
try:
    x5.movj(handle, JP1)
    x5.wait_cmd_send_done(handle)
    time.sleep(20)
    x5.movj(handle, JP2)
    x5.wait_cmd_send_done(handle)
except x5.RobException as ex:
    print(f"关节运动执行失败！")
    print(f"错误代码：{ex.error_code}")
    print(f"错误信息：{ex.error_message}")

# 等待一段时间
time.sleep(MAX_WAIT_INTERVAL_MS / 1000.0)

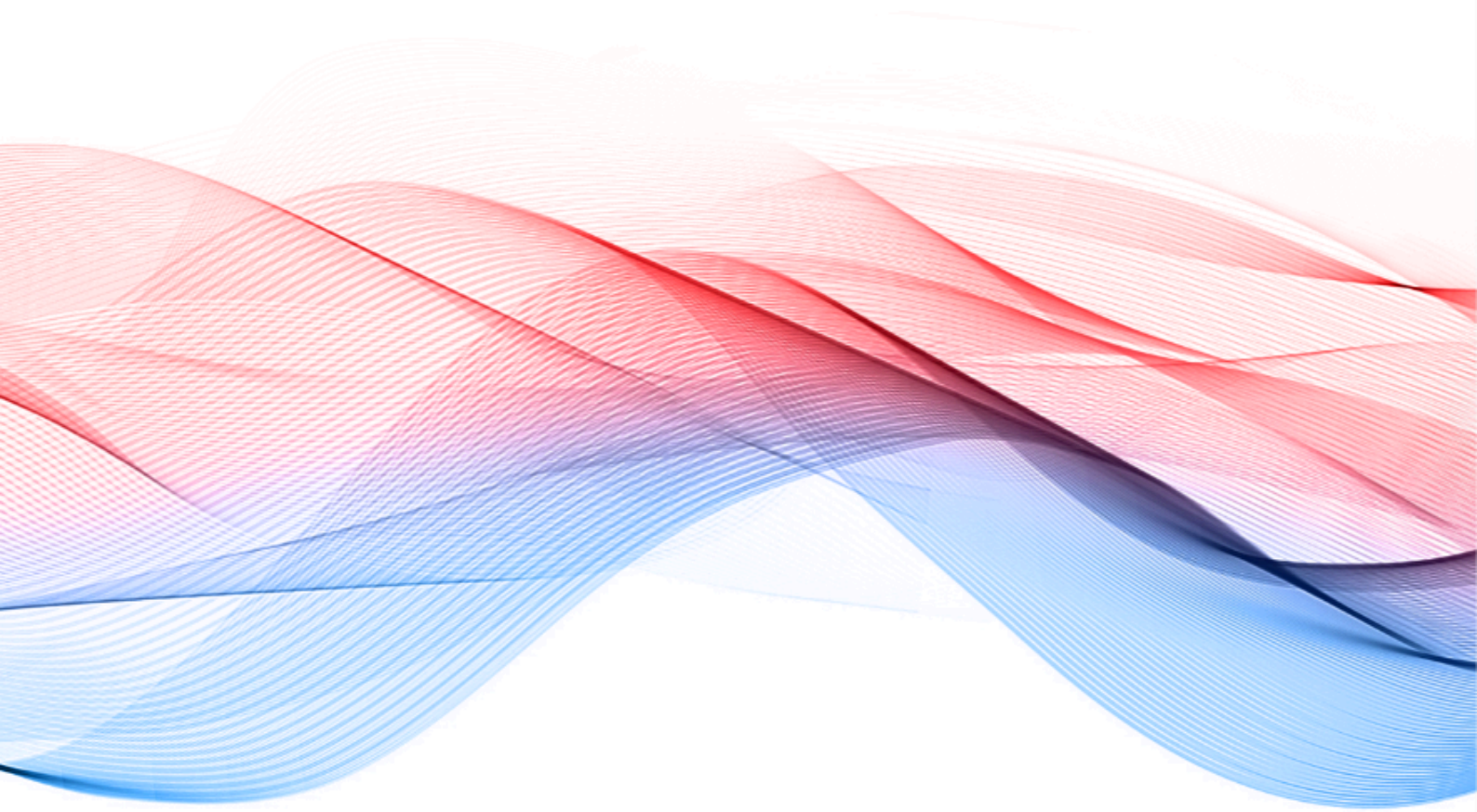
# 释放资源
release_resources(handle)
print("***结束***")
```

报错详情

错误码	错误原因
1000	系统标定计算成功，但倾角超过 10 度
-96	内部指针异常
-97	内存分配失败
-98	命令下发超时
-99	网络连接中断
-100	接口参数越界
-101	接口参数格式错误
-102	接口参数指针异常
-103	附加参数越界
-104	接口含有不支持的运动附加参数
-106	附加参数组合使用异常
-108	笛卡尔点位参数越界
-110	接口参数含有不支持的 Lua 命令
-112	接口参数的 Lua 命令格式错误
-120	接口参数的关节行程超限
-200	重复连接到同一控制器 IP
-201	连接数量已达到上限
-202	句柄值不合法
-300	与控制器版本不匹配
-301	XAPI 与控制器类型(型号)不匹配
-400	程序文件未编译
-401	程序编译中
-402	程序文件名为空
-403	程序文件名后缀错误
-404	程序文件不存在
-405	Lua 程序静态检查失败
-406	程序预读行号异常
-407	程序预读指针丢失
-500	文件读取不完整
-501	程序文件名为空
-502	文件读取失败
-503	文件信息获取失败

错误码	错误原因
-600	机器人当前未处于正确的模式
-601	机器人当前未处于正确的使能状态
-602	机器人当前未处于正确的运行状态
-603	机器人当前未处于正确的远程状态
-604	机器人当前位处于轴到位状态
-605	机器人当前未处于正确的视觉类型状态
-606	机器人当前处于实时引导运动模式(RGM)
-1000	系统周期状态获取失败
-1001	系统周期命令执行失败
-1002	系统变量获取失败
-1003	系统变量写入失败
-1004	系统参数获取失败
-1005	系统参数写入失败
-1006	系统文件获取失败
-1007	系统文件写入失败
-1008	系统时间获取失败
-1009	系统 ECAT(节点)获取失败
-1010	系统 DH 参数获取失败
-1011	系统机器参数获取失败
-1012	系统版本获取失败
-1013	系统内核版本获取失败
-1014	系统资源挂载失败
-1015	系统挂载流程未初始化
-1016	系统资源卸载失败
-1017	系统文件内容错误
-1018	系统文件删除失败
-1019	系统文件打开失败
-1020	系统功能函数调用类型错误
-1021	系统运动学计算失败
-1040	系统 ROBHAL 创建失败
-1041	系统通信初始化失败
-1042	系统通信清理失败
-1050	系统指针设置失败
-1051	系统解释器启动失败

错误码	错误原因
-1052	系统解释器停止失败
-1053	系统解释器恢复类型设置失败
-1100	数据库文件获取失败
-1101	数据库文件写入失败
-1102	数据库操作对象创建失败
-1103	sqlite3_open 失败
-1104	sqlite3_prepare 失败
-1105	伺服类型获取失败
-1200	系统 UDP 通信异常
-1300	系统模块信息获取失败
-3000	文件压缩失败
-3001	文件解压失败
-3100	系统标定读后状态码异常
-3101	系统标定写后状态码异常
-3102	系统标定计算结果错误
-3103	系统标定误差大于 5.0 / 手眼标定(眼在手外)误差大于 50



内容如有更新，恕不另行通知

如想了解拓斯达相关产品，可在拓斯达官方网站查询：

<https://www.topstarltd.com/>

如果您对产品有任何疑问，可通过以下方式联系我们：

售后热线：400-096-8005

